Universidade de Brasilia

Brasília Military Middle School 2017

Arthur Botelho, Henrique Quirino, Luiz Eduardo Reis

2026-08-12

Contest (1)

template.cpp
28 lines

```cpp
#include <bits/stdc++.h>
using namespace std;
// Ours
#define endl '\n'
#define debug(x...) cout<<"["#x"]: ",[](auto...$){((cout<<$<<";
     "),...);}(x),cout<<endl
// KACTL
typedef long long ll;
typedef long double ld;
#define sz(x) (int)x.size()
#define all(x) x.begin(), x.end()
#define vi vector<int>
#define pii pair<int, int>
#define rep(i, a, b) for(int i = (a); i < (b); i++)
#define pb push_back
#define eb emplace_back

constexpr int oo = (((unsigned int)-1)>>2);

void solve(){
  int n; cin >> n;
}

int main() {
  cin.tie(0)->sync_with_stdio(0);
  int t = 1;
  // cin >> t;
  while (t--) solve();
}
```

.bashrc
21 lines

```bash
teste() {
    for f in "$2"/*; do
        echo "=== $f"
        cat $f
        echo "=== Out"
        $1 < $f
    done
}
# teste ./b bf
stress() {
    for ((i=1; ; i++)) do
        echo "Test $i"
        echo $i | $1 > gen_in
        $2 < gen_in > main_out
        $3 < gen_in > brute_out
        if ! diff -w main_out brute_out > /dev/null; then
            return 1
        fi
    done
}
# stress "python3 gen.py" ./main ./brute
```

.vimrc
5 lines

```vim
set autoindent smartindent ts=4 sw=4 rnu nu
" Select region and then type :Hash to hash your selection.
" Useful for verifying that there aren't mistypes.
ca Hash w !cpp -dD -P -fpreprocessed \| tr -d '[:space:]' \
 \| md5sum \| cut -c-6
```

hash.sh
3 lines

```bash
# Hashes a file, ignoring all whitespace and comments. Use for
# verifying that code was correctly typed.
cpp -dD -P -fpreprocessed | tr -d '[:space:]'| md5sum |cut -c-6
```

Makefile
1 lines

```make
CXXFLAGS := -std=c++20 -Wall -Wextra -Wshadow -Wconversion -
    fsanitize=address,undefined -g3
```

Theory (2)

2.1 Geometry

2.1.1 Trigonometry

$$(V+W)\tan(v-w)/2 = (V-W)\tan(v+w)/2$$

where V, W are lengths of sides opposite angles v, w.

$$a\cos x + b\sin x = r\cos(x-\phi)$$
$$a\sin x + b\cos x = r\sin(x+\phi)$$

where $r = \sqrt{a^2+b^2}, \phi = \operatorname{atan2}(b, a)$.

2.1.2 Triangles

Length of median (divides triangle into two equal-area triangles): $m_a = \frac{1}{2}\sqrt{2b^2+2c^2-a^2}$

Length of bisector (divides angles in two):

$$s_a = \sqrt{bc\left[1-\left(\frac{a}{b+c}\right)^2\right]}$$

Law of tangents: $\dfrac{a+b}{a-b} = \dfrac{\tan\frac{\alpha+\beta}{2}}{\tan\frac{\alpha-\beta}{2}}$

2.1.3 Quadrilaterals

With side lengths a, b, c, d, diagonals e, f, diagonals angle θ, area A and magic flux $F = b^2+d^2-a^2-c^2$:

$$4A = 2ef\cdot\sin\theta = F\tan\theta = \sqrt{4e^2f^2-F^2}$$

For cyclic quadrilaterals the sum of opposite angles is $180°$, $ef = ac+bd$, and $A = \sqrt{(p-a)(p-b)(p-c)(p-d)}$.

2.2 Combinatorics

2.2.1 Permutations

Cycles

Let $g_S(n)$ be the number of n-permutations whose cycle lengths all belong to the set S. Then

$$\sum_{n=0}^{\infty} g_S(n)\frac{x^n}{n!} = \exp\left(\sum_{n\in S}\frac{x^n}{n}\right)$$

Derangements

Permutations of a set such that none of the elements appear in their original position.

$$D(n) = (n-1)(D(n-1)+D(n-2)) = nD(n-1)+(-1)^n = \left\lfloor\frac{n!}{e}\right\rceil$$

2.2.2 Partitions and subsets

Partition function

Number of ways of writing n as a sum of positive integers, disregarding the order of the summands.

$$p(0) = 1,\ p(n) = \sum_{k\in\mathbb{Z}\setminus\{0\}}(-1)^{k+1}p(n-k(3k-1)/2)$$

$$p(n) \sim 0.145/n\cdot\exp(2.56\sqrt{n})$$

n	0	1	2	3	4	5	6	7	8	9	20	50	100
$p(n)$	1	1	2	3	5	7	11	15	22	30	627	~2e5	~2e8

Lucas' Theorem

Let n, m be non-negative integers and p a prime. Write $n = n_kp^k+...+n_1p+n_0$ and $m = m_kp^k+...+m_1p+m_0$. Then $\binom{n}{m} \equiv \prod_{i=0}^{k}\binom{n_i}{m_i} \pmod{p}$.

2.2.3 General purpose numbers

Bernoulli numbers

EGF of Bernoulli numbers is $B(t) = \frac{t}{e^t-1}$ (FFT-able). $B[0,\ldots] = [1, -\frac{1}{2}, \frac{1}{6}, 0, -\frac{1}{30}, 0, \frac{1}{42}, \ldots]$

Sums of powers:

$$\sum_{i=1}^{n} n^m = \frac{1}{m+1}\sum_{k=0}^{m}\binom{m+1}{k}B_k\cdot(n+1)^{m+1-k}$$

Euler-Maclaurin formula for infinite sums:

$$\sum_{i=m}^{\infty} f(i) = \int_m^{\infty} f(x)dx - \sum_{k=1}^{\infty} \frac{B_k}{k!} f^{(k-1)}(m)$$
$$\approx \int_m^{\infty} f(x)dx + \frac{f(m)}{2} - \frac{f'(m)}{12} + \frac{f'''(m)}{720} + O(f^{(5)}(m))$$

Stirling numbers of the first kind

Number of permutations on n items with k cycles.

$$c(n,k) = c(n-1,k-1) + (n-1)c(n-1,k),\ \ c(0,0) = 1$$
$$\textstyle\sum_{k=0}^n c(n,k)x^k = x(x+1)\dots(x+n-1)$$

$c(8,k) = 8, 0, 5040, 13068, 13132, 6769, 1960, 322, 28, 1$
$c(n,2) = 0, 0, 1, 3, 11, 50, 274, 1764, 13068, 109584, \dots$

Eulerian numbers

Number of permutations $\pi \in S_n$ in which exactly k elements are greater than the previous element. k j:s s.t. $\pi(j) > \pi(j+1)$, $k+1$ j:s s.t. $\pi(j) \geq j$, k j:s s.t. $\pi(j) > j$.

$$E(n,k) = (n-k)E(n-1,k-1) + (k+1)E(n-1,k)$$
$$E(n,0) = E(n,n-1) = 1$$
$$E(n,k) = \sum_{j=0}^{k} (-1)^j \binom{n+1}{j} (k+1-j)^n$$

Stirling numbers of the second kind

Partitions of n distinct elements into exactly k groups.

$$S(n,k) = S(n-1,k-1) + kS(n-1,k)$$
$$S(n,1) = S(n,n) = 1$$
$$S(n,k) = \frac{1}{k!} \sum_{j=0}^{k} (-1)^{k-j} \binom{k}{j} j^n$$

Bell numbers

Total number of partitions of n distinct elements. $B(n) = 1, 1, 2, 5, 15, 52, 203, 877, 4140, 21147, \dots$. For p prime,

$$B(p^m + n) \equiv mB(n) + B(n+1) \pmod p$$

2.3 Number Theory

2.3.1 Highly composite numbers

Up to: number of divisors (number itself)

$10^2 : 12\ (60)$
$10^3 : 32\ (840)$
$10^4 : 64\ (7560)$
$10^5 : 128\ (83160)$
$10^6 : 240\ (720720)$
$10^7 : 448\ (8648640)$
$10^8 : 768\ (73513440)$
$10^9 : 1344\ (735134400)$
$10^{10} : 2304\ (6983776800)$
$10^{11} : 4032\ (97772875200)$
$10^{12} : 6720\ (963761198400)$
$10^{13} : 10752\ (9316358251200)$
$10^{14} : 17280\ (97821761637600)$
$10^{15} : 26880\ (866421317361600)$
$10^{16} : 41472\ (8086598962041600)$
$10^{17} : 64512\ (74801040398884800)$
$10^{18} : 103680\ (897612484786617600)$

2.4 Graphs

2.4.1 Erdős–Gallai theorem

A simple graph with node degrees $d_1 \geq \dots \geq d_n$ exists iff $d_1 + \dots + d_n$ is even and for every $k = 1 \dots n$,

$$\sum_{i=1}^{k} d_i \leq k(k-1) + \sum_{i=k+1}^{n} \min(d_i, k).$$

2.4.2 Maximum Independent Set

Just find a max clique of the complement. If the graph is bipartite, see MinimumVertexCover.h.

2.4.3 Min Cut

After running a max flow, the left side of a min-cut from s to t is given by all vertices reachable from s, only traversing edges with positive residual capacity.

Math (3)

3.1 Number Theory

3.1.1 Modular Arithmetic

ModMulLL.h

Description: Calculate $a \cdot b \bmod c$ (or $a^b \bmod c$) for $0 \leq a, b \leq c \leq 7.2 \cdot 10^{18}$.
Time: $\mathcal{O}(1)$ for `modmul`, $\mathcal{O}(\log b)$ for `modpow` bbbd8f, 11L, 2m

```
typedef unsigned long long ull;
ull modmul(ull a, ull b, ull M) {
  ll ret = a * b - M * ull(1.L / M * a * b);
  return ret + M * (ret < 0) - M * (ret >= (ll)M);
} // e9309c
ull modpow(ull b, ull e, ull mod) {
  ull ans = 1;
  for (; e; b = modmul(b, b, mod), e /= 2)
    if (e & 1) ans = modmul(ans, b, mod);
  return ans;
} // 100b91
```

ModLog.h

Description: Returns the smallest $x > 0$ s.t. $a^x = b \pmod m$, or -1 if no such x exists. modLog(a,1,m) can be used to calculate the order of a.
Time: $\mathcal{O}(\sqrt{m})$ c040b8, 11L, 2m10s

```
ll modLog(ll a, ll b, ll m) {
  ll n = (ll) sqrt(m) + 1, e = 1, f = 1, j = 1;
  unordered_map<ll, ll> A;
  while (j <= n && (e = f = e * a % m) != b % m)
    A[e * b % m] = j++;
  if (e == b % m) return j;
  if (__gcd(m, e) == __gcd(m, b))
    rep(i,2,n+2) if (A.count(e = e * f % m))
      return n * i - A[e];
  return -1;
} // c040b8
```

ModSqrt.h

Description: Tonelli-Shanks algorithm for modular square roots. Finds x s.t. $x^2 = a \pmod p$ ($-x$ gives the other solution).
Time: $\mathcal{O}(\log^2 p)$ worst case, $\mathcal{O}(\log p)$ for most p

"ModPow.h" 439cc2, 21L, 4m20s

```
const ll p = mod; // mod is p
ll sqrt(ll a) {
  a %= p; if (a < 0) a += p;
  if (a == 0) return 0;
  assert(modpow(a, (p-1)/2) == 1); // else no solution
  if (p % 4 == 3) return modpow(a, (p+1)/4);
  // a^(n+3)/8 or 2^(n+3)/8 * 2^(n-1)/4 works if p % 8 == 5
  ll s = p - 1, n = 2;
  int r = 0, m;
  while (s % 2 == 0) ++r, s /= 2;
  while (modpow(n, (p - 1) / 2) != p - 1) ++n;
  ll x = modpow(a, (s + 1) / 2);
  ll b = modpow(a, s), g = modpow(n, s);
  for (;; r = m) {
    ll t = b;
    for (m = 0; m < r && t != 1; ++m) t = t * t % p;
    if (m == 0) return x;
    ll gs = modpow(g, 1LL << (r - m - 1));
    g = gs * gs % p; x = x * gs % p; b = b * g % p;
  } // d569f9
} // c86781
```

CRT.h

Description: Chinese Remainder Theorem.
`crt(a, m, b, n)` computes x such that $x \equiv a \pmod m$, $x \equiv b \pmod n$. If $|a| < m$ and $|b| < n$, x will obey $0 \leq x < \mathrm{lcm}(m,n)$. Assumes $mn < 2^{62}$.
Time: $\log(n)$

"euclid.h" 04d93a, 7L, 1m30s

```
ll crt(ll a, ll m, ll b, ll n) {
  if (n > m) swap(a, b), swap(m, n);
  ll x, y, g = euclid(m, n, x, y);
  assert((a - b) % g == 0); // else no solution
  x = (b - a) % n * x % n / g * m + a;
  return x < 0 ? x + m*n/g : x;
} // 04d93a
```

3.1.2 Primality and Divisibility

FastEratosthenes.h

Description: Prime sieve for generating all primes smaller than LIM.
Time: LIM=1e9 $\approx$ 1.5s

6b2912, 20L, 4m

```
const int LIM = 1e6;
bitset<LIM> isPrime;
vi eratosthenes() {
  const int S = (int)round(sqrt(LIM)), R = LIM / 2;
  vi pr = {2}, sieve(S+1); pr.reserve(int(LIM/log(LIM)*1.1));
  vector<pii> cp;
  for (int i = 3; i <= S; i += 2) if (!sieve[i]) {
    cp.push_back({i, i * i / 2});
    for (int j = i * i; j <= S; j += 2 * i) sieve[j] = 1;
  } // d22e52
  for (int L = 1; L <= R; L += S) {
    array<bool, S> block{};
    for (auto &[p, idx] : cp)
      for (int i=idx; i < S+L; idx = (i+=p)) block[i-L] = 1;
    rep(i,0,min(S, R - L))
      if (!block[i]) pr.push_back((L + i) * 2 + 1);
  } // 5b6623
  for (int i : pr) isPrime[i] = 1;
  return pr;
} // 8ee6d2
```

MillerRabin.h

Description: Deterministic Miller-Rabin primality test. Guaranteed to work for numbers up to $7 \cdot 10^{18}$; for larger numbers, use Python and extend A randomly.
Time: 7 times the complexity of $a^b \mod c$.

"ModMulLL.h" 60dcd1, 12L, 2m30s

```
bool isPrime(ull n) {
  if (n < 2 || n % 6 % 4 != 1) return (n | 1) == 3;
  ull A[] = {2, 325, 9375, 28178, 450775, 9780504, 1795265022},
      s = __builtin_ctzll(n-1), d = n >> s;
  for (ull a : A) {   // ^ count trailing zeroes
    ull p = modpow(a%n, d, n), i = s;
    while (p != 1 && p != n - 1 && a % n && i--)
      p = modmul(p, p, n);
    if (p != n-1 && i != s) return 0;
  } // edfaf1
  return 1;
} // 60dcd1
```

PollardRho.h

Description: Pollard-rho randomized factorization algorithm. Returns prime factors of a number, in arbitrary order (e.g. 2299 -> {11, 19, 11}).
Time: $\mathcal{O}\left(n^{1/4}\right)$, less for numbers with small factors.

"ModMulLL.h", "MillerRabin.h" d8d98d, 18L, 3m20s

```
ull pollard(ull n) {
  ull x = 0, y = 0, t = 30, prd = 2, i = 1, q;
  auto f = [&](ull x) { return modmul(x, x, n) + i; };
  while (t++ % 40 || __gcd(prd, n) == 1) {
    if (x == y) x = ++i, y = f(x);
    if ((q = modmul(prd, max(x,y) - min(x,y), n))) prd = q;
    x = f(x), y = f(f(y));
  } // 989d40
  return __gcd(prd, n);
} // cd2ac3
vector<ull> factor(ull n) {
  if (n == 1) return {};
  if (isPrime(n)) return {n};
  ull x = pollard(n);
  auto l = factor(x), r = factor(n / x);
  l.insert(l.end(), all(r));
  return l;
} // d54ba8
```

euclid.h

Description: Finds two integers x and y, such that $ax + by = \gcd(a, b)$. If you just need gcd, use the built in __gcd instead. If a and b are coprime, then x is the inverse of $a \pmod{b}$.

33ba8f, 5L, 50s

```
ll euclid(ll a, ll b, ll &x, ll &y) {
  if (!b) return x = 1, y = 0, a;
  ll d = euclid(b, a % b, y, x);
  return y -= a/b * x, d;
} // 33ba8f
```

3.2 Discrete Math

3.2.1 Convolutions

FastFourierTransform.h

Description: fft(a) computes $\hat{f}(k) = \sum_x a[x] \exp(2\pi i \cdot kx/N)$ for all k. N must be a power of 2. Useful for convolution: conv(a, b) = c, where $c[x] = \sum a[i]b[x - i]$. For convolution of complex numbers or more than two vectors: FFT, multiply pointwise, divide by n, reverse(start+1, end), FFT back. Rounding is safe if $(\sum a_i^2 + \sum b_i^2) \log_2 N < 9 \cdot 10^{14}$ (in practice 10^{16}; higher for random inputs). Otherwise, use NTT/FFTMod.
Time: $\mathcal{O}(N \log N)$ with $N = |A| + |B|$ ($\sim$1s for $N = 2^{22}$)

3dd197, 36L, 7m50s

```
typedef complex<double> C;
typedef vector<double> vd;
void fft(vector<C>& a) {
  int n = sz(a), L = 31 - __builtin_clz(n);
  static vector<complex<long double>> R(2, 1); // (10% faster
      if double)
  static vector<C> rt(2, 1);
  for (static int k = 2; k < n; k *= 2) {
    R.resize(n); rt.resize(n);
    auto x = polar(1.0L, acos(-1.0L) / k);
    rep(i,k,2*k) rt[i] = R[i] = i&1 ? R[i/2] * x : R[i/2];
  } // a8a74e
  vi rev(n);
  rep(i,0,n) rev[i] = (rev[i / 2] | (i & 1) << L) / 2;
  rep(i,0,n) if (i < rev[i]) swap(a[i], a[rev[i]]);
  for (int k = 1; k < n; k *= 2)
    for (int i = 0; i < n; i += 2 * k) rep(j,0,k) {
      auto x = (double *)&rt[j+k], y = (double *)&a[i+j+k];
      C z(x[0]*y[0] - x[1]*y[1], x[0]*y[1] + x[1]*y[0]);
      a[i + j + k] = a[i + j] - z;
      a[i + j] += z;
    } // 69f11b
} // 2258f7
vd conv(const vd& a, const vd& b) {
  if (a.empty() || b.empty()) return {};
  vd res(sz(a) + sz(b) - 1);
  int L = 32 - __builtin_clz(sz(res)), n = 1 << L;
  vector<C> in(n), out(n);
  copy(all(a), begin(in));
  rep(i,0,sz(b)) in[i].imag(b[i]);
  fft(in);
  for (C& x : in) x *= x;
  rep(i,0,n) out[i] = in[-i & (n - 1)] - conj(in[i]);
  fft(out);
  rep(i,0,sz(res)) res[i] = imag(out[i]) / (4 * n);
  return res;
} // 873509
```

FastFourierTransformMod.h

Description: Higher precision FFT, can be used for convolutions modulo arbitrary integers as long as $N \log_2 N \cdot \text{mod} < 8.6 \cdot 10^{14}$ (in practice 10^{16} or higher). Inputs must be in $[0, \text{mod})$.
Time: $\mathcal{O}(N \log N)$, where $N = |A| + |B|$ (twice as slow as NTT or FFT)

"FastFourierTransform.h" b82773, 22L, 5m

```
typedef vector<ll> vl;
template<int M> vl convMod(const vl &a, const vl &b) {
  if (a.empty() || b.empty()) return {};
  vl res(sz(a) + sz(b) - 1);
  int B=32-__builtin_clz(sz(res)), n=1<<B, cut=int(sqrt(M));
  vector<C> L(n), R(n), outs(n), outl(n);
  rep(i,0,sz(a)) L[i] = C((int)a[i] / cut, (int)a[i] % cut);
  rep(i,0,sz(b)) R[i] = C((int)b[i] / cut, (int)b[i] % cut);
  fft(L), fft(R);
  rep(i,0,n) {
    int j = -i & (n - 1);
    outl[j] = (L[i] + conj(L[j])) * R[i] / (2.0 * n);
    outs[j] = (L[i] - conj(L[j])) * R[i] / (2.0 * n) / 1i;
  } // cb3017
  fft(outl), fft(outs);
  rep(i,0,sz(res)) {
    ll av = ll(real(outl[i])+.5), cv = ll(imag(outs[i])+.5);
    ll bv = ll(imag(outl[i])+.5) + ll(real(outs[i])+.5);
    res[i] = ((av % M * cut + bv) % M * cut + cv) % M;
  } // 58fa4f
  return res;
} // c1f2f1
```

NumberTheoreticTransform.h

Description: ntt(a) computes $\hat{f}(k) = \sum_x a[x] g^{xk}$ for all k, where $g = \text{root}^{(mod-1)/N}$. N must be a power of 2. Useful for convolution modulo specific nice primes of the form $2^a b + 1$, where the convolution result has size at most 2^a. For arbitrary modulo, see FFTMod. conv(a, b) = c, where $c[x] = \sum a[i]b[x - i]$. For manual convolution: NTT the inputs, multiply pointwise, divide by n, reverse(start+1, end), NTT back. Inputs must be in [0, mod).
Time: $\mathcal{O}(N \log N)$

"../math/ModPow.h" a46443, 35L, 7m50s

```
const ll root = 62; // change mod to (119 << 23) + 1 =
    998244353
// For p < 2^30 there is also e.g. 5 << 25, 7 << 26, 479 << 21
// and 483 << 21 (same root). The last two are > 10^9.
typedef vector<ll> vl;
void ntt(vl &a) {
  int n = sz(a), L = 31 - __builtin_clz(n);
  static vl rt(2, 1);
  for (static int k = 2, s = 2; k < n; k *= 2, s++) {
    rt.resize(n);
    ll z[] = {1, modpow(root, mod >> s)};
    rep(i,k,2*k) rt[i] = rt[i / 2] * z[i & 1] % mod;
  } // f39127
  vi rev(n);
  rep(i,0,n) rev[i] = (rev[i / 2] | (i & 1) << L) / 2;
  rep(i,0,n) if (i < rev[i]) swap(a[i], a[rev[i]]);
  for (int k = 1; k < n; k *= 2)
    for (int i = 0; i < n; i += 2 * k) rep(j,0,k) {
      ll z = rt[j + k] * a[i + j + k] % mod, &ai = a[i + j];
      a[i + j + k] = ai - z + (z > ai ? mod : 0);
      ai += (ai + z >= mod ? z - mod : z);
    } // 9a8565
} // 3b763b
vl conv(const vl &a, const vl &b) {
  if (a.empty() || b.empty()) return {};
  int s = sz(a) + sz(b) - 1, B = 32 - __builtin_clz(s),
      n = 1 << B;
  int inv = modpow(n, mod - 2);
  vl L(a), R(b), out(n);
  L.resize(n), R.resize(n);
  ntt(L), ntt(R);
  rep(i,0,n)
    out[-i & (n - 1)] = (ll)L[i] * R[i] % mod * inv % mod;
```

```
  ntt(out);
  return {out.begin(), out.begin() + s};
} // 3876bf
```

FastSubsetTransform.h

Description: Transform to a basis with fast convolutions of the form $c[z] = \sum_{z=x\oplus y} a[x] \cdot b[y]$, where $\oplus$ is one of AND, OR, XOR. The size of a must be a power of two.
Time: $\mathcal{O}(N \log N)$

464cf3, 16L, 3m20s

```
void FST(vi& a, bool inv) {
  for (int n = sz(a), step = 1; step < n; step *= 2) {
    for (int i = 0; i < n; i += 2 * step) rep(j,i,i+step) {
      int &u = a[j], &v = a[j + step]; tie(u, v) =
        inv ? pii(v - u, u) : pii(v, u + v); // AND
        inv ? pii(v, u - v) : pii(u + v, u); // OR
        pii(u + v, u - v);                   // XOR
    } // 7b7e71
  } // faec61
  if (inv) for (int& x : a) x /= sz(a); // XOR only
} // 57eeaf
vi conv(vi a, vi b) {
  FST(a, 0); FST(b, 0);
  rep(i,0,sz(a)) a[i] *= b[i];
  FST(a, 1); return a;
} // 3cbd48
```

3.2.2 Recurrences and Sums

BerlekampMassey.h

Description: Recovers any n-order linear recurrence relation from the first $2n$ terms of the recurrence. Useful for guessing linear recurrences after brute-forcing the first terms. Should work on any field, but numerical stability for floats is not guaranteed. Output will have size $\leq n$.
Usage: `berlekampMassey({0, 1, 1, 3, 5, 11}) // {1, 2}`
Time: $\mathcal{O}(N^2)$

"../math/ModPow.h" 96548b, 17L, 3m20s

```
vector<ll> berlekampMassey(vector<ll> s) {
  int n = sz(s), L = 0, m = 0;
  vector<ll> C(n), B(n), T;
  C[0] = B[0] = 1; ll b = 1;
  rep(i,0,n) { ++m;
    ll d = s[i] % mod;
    rep(j,1,L+1) d = (d + C[j] * s[i - j]) % mod;
    if (!d) continue;
    T = C; ll coef = d * modpow(b, mod-2) % mod;
    rep(j,m,n) C[j] = (C[j] - coef * B[j - m]) % mod;
    if (2 * L > i) continue;
    L = i + 1 - L; B = T; b = d; m = 0;
  } // 8c2376
  C.resize(L + 1); C.erase(C.begin());
  for (ll& x : C) x = (mod - x) % mod;
  return C;
} // 96548b
```

LinearRecurrence.h

Description: Generates the k'th term of an n-order linear recurrence $S[i] = \sum_j S[i-j-1]tr[j]$, given $S[0 \ldots \geq n-1]$ and $tr[0 \ldots n-1]$. Faster than matrix multiplication. Useful together with Berlekamp–Massey.
Usage: `linearRec({0, 1}, {1, 1}, k) // k'th Fibonacci number`
Time: $\mathcal{O}(n^2 \log k)$

f4e444, 21L, 3m50s

```
typedef vector<ll> Poly;
ll linearRec(Poly S, Poly tr, ll k) {
  int n = sz(tr);
  auto combine = [&](Poly a, Poly b) {
    Poly res(n * 2 + 1);
    rep(i,0,n+1) rep(j,0,n+1)
      res[i + j] = (res[i + j] + a[i] * b[j]) % mod;
    for (int i = 2 * n; i > n; --i) rep(j,0,n)
      res[i - 1 - j] = (res[i - 1 - j] + res[i] * tr[j]) % mod;
    res.resize(n + 1);
    return res;
  }; // 55c8ab
  Poly pol(n + 1), e(pol); pol[0] = e[1] = 1;
  for (++k; k; k /= 2) {
    if (k % 2) pol = combine(pol, e);
    e = combine(e, e);
  } // 8137be
  ll res = 0;
  rep(i,0,n) res = (res + pol[i + 1] * S[i]) % mod;
  return res;
} // 5948dc
```

FloorModSum.h

Description: Floor sum and Mod sum.
`modsum(to, c, k, m)` $= \sum_{i=0}^{to-1} (ki+c)\%m$. `divsum` is similar but for floored division.
Time: $\log(m)$, with a large constant.

5c5bc5, 15L, 3m20s

```
typedef unsigned long long ull;
// written in a weird way to deal with overflows correctly
ull sumsq(ull to) { return to / 2 * ((to-1) | 1); }
ull divsum(ull to, ull c, ull k, ull m) {
  ull res = k / m * sumsq(to) + c / m * to;
  k %= m; c %= m;
  if (!k) return res;
  ull to2 = (to * k + c) / m;
  return res + (to - 1) * to2 - divsum(to2, m-1 - c, m, k);
} // 78bfc8
ll modsum(ull to, ll c, ll k, ll m) {
  c = ((c % m) + m) % m;
  k = ((k % m) + m) % m;
  return to * c + k * sumsq(to) - m * divsum(to, c, k, m);
} // 5daf3e
```

3.3 Linear Algebra

3.3.1 Matrices

Determinant.h

Description: Calculates determinant of a matrix. Destroys the matrix.
Time: $\mathcal{O}(N^3)$

bd5cec, 15L, 2m30s

```
double det(vector<vector<double>>& a) {
  int n = sz(a); double res = 1;
  rep(i,0,n) {
    int b = i;
    rep(j,i+1,n) if (fabs(a[j][i]) > fabs(a[b][i])) b = j;
    if (i != b) swap(a[i], a[b]), res *= -1;
    res *= a[i][i];
    if (res == 0) return 0;
    rep(j,i+1,n) {
      double v = a[j][i] / a[i][i];
      if (v != 0) rep(k,i+1,n) a[j][k] -= v * a[i][k];
    } // 4ec6a2
  } // ee1466
  return res;
} // bd5cec
```

IntDeterminant.h

Description: Calculates determinant using modular arithmetics. Modulos can also be removed to get a pure-integer version.
Time: $\mathcal{O}(N^3)$

3313dc, 18L, 2m30s

```
const ll mod = 12345;
ll det(vector<vector<ll>>& a) {
  int n = sz(a); ll ans = 1;
  rep(i,0,n) {
    rep(j,i+1,n) {
      while (a[j][i] != 0) { // gcd step
        ll t = a[i][i] / a[j][i];
        if (t) rep(k,i,n)
          a[i][k] = (a[i][k] - a[j][k] * t) % mod;
        swap(a[i], a[j]);
        ans *= -1;
      } // e81b29
    } // 30d1b2
    ans = ans * a[i][i] % mod;
    if (!ans) return 0;
  } // f39a45
  return (ans + mod) % mod;
} // 5e85f0
```

MatrixInverse.h

Description: Invert matrix A. Returns rank; result is stored in A unless singular (rank < n). Can easily be extended to prime moduli; for prime powers, repeatedly set $A^{-1} = A^{-1}(2I - AA^{-1}) \pmod{p^k}$ where A^{-1} starts as the inverse of A mod p, and k is doubled in each step.
Time: $\mathcal{O}(n^3)$

ebfff6, 32L, 5m40s

```
int matInv(vector<vector<double>>& A) {
  int n = sz(A); vi col(n);
  vector<vector<double>> tmp(n, vector<double>(n));
  rep(i,0,n) tmp[i][i] = 1, col[i] = i;
  rep(i,0,n) {
    int r = i, c = i;
    rep(j,i,n) rep(k,i,n)
      if (fabs(A[j][k]) > fabs(A[r][c]))
        r = j, c = k;
    if (fabs(A[r][c]) < 1e-12) return i;
    A[i].swap(A[r]); tmp[i].swap(tmp[r]);
    rep(j,0,n)
      swap(A[j][i], A[j][c]), swap(tmp[j][i], tmp[j][c]);
    swap(col[i], col[c]);
    double v = A[i][i];
    rep(j,i+1,n) {
      double f = A[j][i] / v;
      A[j][i] = 0;
      rep(k,i+1,n) A[j][k] -= f*A[i][k];
      rep(k,0,n) tmp[j][k] -= f*tmp[i][k];
    } // ebeea3
    rep(j,i+1,n) A[i][j] /= v;
    rep(j,0,n) tmp[i][j] /= v;
    A[i][i] = 1;
  } // 26d90b
  for (int i = n-1; i > 0; --i) rep(j,0,i) {
    double v = A[j][i];
    rep(k,0,n) tmp[j][k] -= v*tmp[i][k];
  } // 03ae0c
  rep(i,0,n) rep(j,0,n) A[col[i]][col[j]] = tmp[i][j];
  return n;
} // ebfff6
```

3.3.2 Systems

SolveLinear.h

Description: Solves $A * x = b$. If there are multiple solutions, an arbitrary one is returned. Returns rank, or -1 if no solutions. Data in A and b is lost.
Time: $\mathcal{O}(n^2 m)$

44c9ab, 35L, 5m30s

```
typedef vector<double> vd;
const double eps = 1e-12;
int solveLinear(vector<vd>& A, vd& b, vd& x) {
  int n = sz(A), m = sz(x), rank = 0, br, bc;
  if (n) assert(sz(A[0]) == m);
  vi col(m); iota(all(col), 0);
  rep(i,0,n) {
    double v, bv = 0;
    rep(r,i,n) rep(c,i,m)
      if ((v = fabs(A[r][c])) > bv)
        br = r, bc = c, bv = v;
    if (bv <= eps) {
      rep(j,i,n) if (fabs(b[j]) > eps) return -1;
      break;
    } // c92205
    swap(A[i], A[br]);
    swap(b[i], b[br]);
    swap(col[i], col[bc]);
    rep(j,0,n) swap(A[j][i], A[j][bc]);
    bv = 1/A[i][i];
    rep(j,i+1,n) {
      double fac = A[j][i] * bv;
      b[j] -= fac * b[i];
      rep(k,i+1,m) A[j][k] -= fac*A[i][k];
    } // 881860
    rank++;
  } // 0f0f0f
  x.assign(m, 0);
  for (int i = rank; i--;) {
    b[i] /= A[i][i];
    x[col[i]] = b[i];
    rep(j,0,i) b[j] -= A[j][i] * b[i];
  } // ed1d08
  return rank; // (multiple solutions if rank < m)
} // ade67b
```

SolveLinear2.h

Description: To get all uniquely determined values of x back from SolveLinear, make the following changes:

"SolveLinear.h" 08e495, 7L, 1m20s

```
rep(j,0,n) if (j != i) // instead of rep(j,i+1,n)
// ... then at the end:
x.assign(m, undefined);
rep(i,0,rank) {
  rep(j,rank,m) if (fabs(A[i][j]) > eps) goto fail;
  x[col[i]] = b[i] / A[i][i];
fail:; } // 87878c
```

SolveLinearBinary.h

Description: Solves $Ax = b$ over $\mathbb{F}_2$. If there are multiple solutions, one is returned arbitrarily. Returns rank, or -1 if no solutions. Destroys A and b.
Time: $\mathcal{O}(n^2 m)$

fa2d7a, 32L, 4m50s

```
typedef bitset<1000> bs;
int solveLinear(vector<bs>& A, vi& b, bs& x, int m) {
  int n = sz(A), rank = 0, br;
  assert(m <= sz(x));
  vi col(m); iota(all(col), 0);
  rep(i,0,n) {
    for (br=i; br<n; ++br) if (A[br].any()) break;
    if (br == n) {
      rep(j,i,n) if(b[j]) return -1;
      break;
    } // 80687c
    int bc = (int)A[br]._Find_next(i-1);
    swap(A[i], A[br]);
    swap(b[i], b[br]);
    swap(col[i], col[bc]);
    rep(j,0,n) if (A[j][i] != A[j][bc]) {
      A[j].flip(i); A[j].flip(bc);
    } // b44a9b
    rep(j,i+1,n) if (A[j][i]) {
      b[j] ^= b[i];
      A[j] ^= A[i];
    } // 87192e
    rank++;
  } // fe9281
  x = bs();
  for (int i = rank; i--;) {
    if (!b[i]) continue;
    x[col[i]] = 1;
    rep(j,0,i) b[j] ^= A[j][i];
  } // 8fdba1
  return rank; // (multiple solutions if rank < m)
} // 26d73e
```

Tridiagonal.h

Description: $x = \text{tridiagonal}(d, p, q, b)$ solves the equation system

$$\begin{pmatrix} b_0 \\ b_1 \\ b_2 \\ b_3 \\ \vdots \\ b_{n-1} \end{pmatrix} = \begin{pmatrix} d_0 & p_0 & 0 & 0 & \cdots & 0 \\ q_0 & d_1 & p_1 & 0 & \cdots & 0 \\ 0 & q_1 & d_2 & p_2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \ddots & \ddots & \vdots \\ 0 & 0 & \cdots & q_{n-3} & d_{n-2} & p_{n-2} \\ 0 & 0 & \cdots & 0 & q_{n-2} & d_{n-1} \end{pmatrix} \begin{pmatrix} x_0 \\ x_1 \\ x_2 \\ x_3 \\ \vdots \\ x_{n-1} \end{pmatrix}.$$

This is useful for solving problems on the type

$$a_i = b_i a_{i-1} + c_i a_{i+1} + d_i,\ 1 \le i \le n,$$

where a_0, a_{n+1}, b_i, c_i and d_i are known. a can then be obtained from

$$\{a_i\} = \text{tridiagonal}(\{1, -1, -1, ..., -1, 1\}, \{0, c_1, c_2, \ldots, c_n\}, \{b_1, b_2, \ldots, b_n, 0\}, \{a_0, d_1, d_2, \ldots, d_n, a_{n+1}\}).$$

Fails if the solution is not unique.
If $|d_i| > |p_i| + |q_{i-1}|$ for all i, or $|d_i| > |p_{i-1}| + |q_i|$, or the matrix is positive definite, the algorithm is numerically stable and neither `tr` nor the check for `diag[i] == 0` is needed.
Time: $\mathcal{O}(N)$

8f9fa8, 26L, 4m20s

```
typedef double T;
vector<T> tridiagonal(vector<T> diag, const vector<T>& super,
    const vector<T>& sub, vector<T> b) {
  int n = sz(b); vi tr(n);
  rep(i,0,n-1) {
    if (abs(diag[i]) < 1e-9 * abs(super[i])) { // diag[i] == 0
      b[i+1] -= b[i] * diag[i+1] / super[i];
      if (i+2 < n) b[i+2] -= b[i] * sub[i+1] / super[i];
      diag[i+1] = sub[i]; tr[++i] = 1;
    } else { // 464c09
      diag[i+1] -= super[i]*sub[i]/diag[i];
      b[i+1] -= b[i]*sub[i]/diag[i];
    } // 62de5a
  } // ed9cce
  for (int i = n; i--;) {
    if (tr[i]) {
      swap(b[i], b[i-1]);
      diag[i-1] = diag[i];
      b[i] /= super[i-1];
    } else { // 0543e4
      b[i] /= diag[i];
      if (i) b[i-1] -= b[i]*super[i-1];
    } // aa91c6
  } // 28af28
  return b;
} // 06d549
```

3.4 Calculus

3.4.1 Functions

Integrate.h

Description: Simple integration of a function over an interval using Simpson's rule. The error should be proportional to h^4, although in practice you will want to verify that the result is stable to desired precision when epsilon changes.

4756fc, 6L, 1m20s

```
template<class F>
double quad(double a, double b, F f, const int n = 1000) {
  double h = (b - a) / 2 / n, v = f(a) + f(b);
  rep(i,1,n*2) v += f(a + i*h) * (i&1 ? 4 : 2);
  return v * h / 3;
} // ddcce2
```

IntegrateAdaptive.h

Description: Fast integration using an adaptive Simpson's rule.
Usage: `double sphereVolume = quad(-1, 1, [](double x) {`
`return quad(-1, 1, [&](double y) {`
`return quad(-1, 1, [&](double z) {`
`return x*x + y*y + z*z < 1; });});});`

92dd79, 12L, 2m50s

```
typedef double d;
#define S(a,b) (f(a) + 4*f((a+b) / 2) + f(b)) * (b-a) / 6
template <class F> d rec(F& f, d a, d b, d eps, d S) {
  d c = (a + b) / 2;
  d S1 = S(a, c), S2 = S(c, b), T = S1 + S2;
  if (abs(T - S) <= 15 * eps || b - a < 1e-10)
    return T + (T - S) / 15;
  return rec(f, a, c, eps / 2, S1) + rec(f, c, b, eps / 2, S2);
} // 10c680
template<class F> d quad(d a, d b, F f, d eps = 1e-8) {
  return rec(f, a, b, eps, S(a, b));
} // 7afc73
```

Simplex.h

Description: Solves a general linear maximization problem: maximize $c^T x$ subject to $Ax \le b$, $x \ge 0$. Returns -inf if there is no solution, inf if there are arbitrarily good solutions, or the maximum value of $c^T x$ otherwise. The input vector is set to an optimal x (or in the unbounded case, an arbitrary solution fulfilling the constraints). Numerical stability is not guaranteed. For better performance, define variables such that $x = 0$ is viable.
Usage: `vvd A = {{1,-1}, {-1,1}, {-1,-2}};`
`vd b = {1,1,-4}, c = {-1,-1}, x;`
`T val = LPSolver(A, b, c).solve(x);`
Time: $\mathcal{O}(NM * \#pivots)$, where a pivot may be e.g. an edge relaxation. $\mathcal{O}(2^n)$ in the general case.

aa8530, 60L, 11m20s

```
typedef double T; // long double, Rational, double + mod<P>...
typedef vector<T> vd;
typedef vector<vd> vvd;
const T eps = 1e-8, inf = 1/.0;
#define MP make_pair
#define ltj(X) if(s == -1 || MP(X[j],N[j]) < MP(X[s],N[s])) s=j
struct LPSolver {
  int m, n; vi N, B; vvd D;
  LPSolver(const vvd& A, const vd& b, const vd& c) :
    m(sz(b)), n(sz(c)), N(n+1), B(m), D(m+2, vd(n+2)) {
      rep(i,0,m) rep(j,0,n) D[i][j] = A[i][j];
      rep(i,0,m) { B[i] = n+i; D[i][n] = -1; D[i][n+1] = b[i];}
      rep(j,0,n) { N[j] = j; D[m][j] = -c[j]; }
```

```
      N[n] = -1; D[m+1][n] = 1;
    } // 6ff8e9
  void pivot(int r, int s) {
    T *a = D[r].data(), inv = 1 / a[s];
    rep(i,0,m+2) if (i != r && abs(D[i][s]) > eps) {
      T *b = D[i].data(), inv2 = b[s] * inv;
      rep(j,0,n+2) b[j] -= a[j] * inv2;
      b[s] = a[s] * inv2;
    } // ca4460
    rep(j,0,n+2) if (j != s) D[r][j] *= inv;
    rep(i,0,m+2) if (i != r) D[i][s] *= -inv;
    D[r][s] = inv;
    swap(B[r], N[s]);
  } // 9cd0a8
  bool simplex(int phase) {
    int x = m + phase - 1;
    for (;;) {
      int s = -1;
      rep(j,0,n+1) if (N[j] != -phase) ltj(D[x]);
      if (D[x][s] >= -eps) return true;
      int r = -1;
      rep(i,0,m) {
        if (D[i][s] <= eps) continue;
        if (r == -1 || MP(D[i][n+1] / D[i][s], B[i])
                     < MP(D[r][n+1] / D[r][s], B[r])) r = i;
      } // 46853f
      if (r == -1) return false;
      pivot(r, s);
    } // 7d839b
  } // f15644
  T solve(vd &x) {
    int r = 0;
    rep(i,1,m) if (D[i][n+1] < D[r][n+1]) r = i;
    if (D[r][n+1] < -eps) {
      pivot(r, n);
      if (!simplex(2) || D[m+1][n+1] < -eps) return -inf;
      rep(i,0,m) if (B[i] == -1) {
        int s = 0;
        rep(j,1,n+1) ltj(D[i]);
        pivot(i, s);
      } // 683310
    } // b6553f
    bool ok = simplex(1); x = vd(n);
    rep(i,0,m) if (B[i] < n) x[B[i]] = D[i][n+1];
    return ok ? D[m][n+1] : inf;
  } // 396a95
}; // c57b35
```

3.4.2 Polynomials

PolyRoots.h

Description: Finds the real roots to a polynomial.
Usage: `polyRoots({{2,-3,1}},-1e9,1e9) // solve x^2-3x+2 = 0`
Time: $\mathcal{O}\left(n^2 \log(1/\epsilon)\right)$

e897c3, 40L, 6m10s

```
struct Poly {
  vector<double> a;
  double operator()(double x) const {
    double val = 0;
    for (int i = sz(a); i--;) (val *= x) += a[i];
    return val;
  } // ae76f3
  void diff() {
    rep(i,1,sz(a)) a[i-1] = i*a[i];
    a.pop_back();
  } // afcaea
  void divroot(double x0) {
    double b = a.back(), c; a.back() = 0;
    for(int i=sz(a)-1; i--;) c = a[i], a[i] = a[i+1]*x0+b, b=c;
    a.pop_back();
  } // 3f874a
}; // c9b7b0
vector<double> polyRoots(Poly p, double xmin, double xmax) {
  if (sz(p.a) == 2) { return {-p.a[0]/p.a[1]}; }
  vector<double> ret;
  Poly der = p;
  der.diff();
  auto dr = polyRoots(der, xmin, xmax);
  dr.push_back(xmin-1);
  dr.push_back(xmax+1);
  sort(all(dr));
  rep(i,0,sz(dr)-1) {
    double l = dr[i], h = dr[i+1];
    bool sign = p(l) > 0;
    if (sign ^ (p(h) > 0)) {
      rep(it,0,60) { // while (h - l > 1e-8)
        double m = (l + h) / 2, f = p(m);
        if ((f <= 0) ^ sign) l = m;
        else h = m;
      } // b69f41
      ret.push_back((l + h) / 2);
    } // fc22f0
  } // d15986
  return ret;
} // b00bfe
```

PolyInterpolate.h

Description: Given n points (x[i], y[i]), computes an n-1-degree polynomial p that passes through them: $p(x) = a[0] * x^0 + \ldots + a[n-1] * x^{n-1}$. For numerical precision, pick $x[k] = c * \cos(k/(n-1) * \pi), k = 0 \ldots n-1$.
Time: $\mathcal{O}\left(n^2\right)$

08bf48, 13L, 2m

```
typedef vector<double> vd;
vd interpolate(vd x, vd y, int n) {
  vd res(n), temp(n);
  rep(k,0,n-1) rep(i,k+1,n)
    y[i] = (y[i] - y[k]) / (x[i] - x[k]);
  double last = 0; temp[0] = 1;
  rep(k,0,n) rep(i,0,n) {
    res[i] += y[k] * temp[i];
    swap(last, temp[i]);
    temp[i] -= last * x[k];
  } // 4c74fe
  return res;
} // 285367
```

Geometry (4)

4.1 Geometric primitives

Point.h

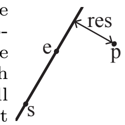
Description: Class to handle points in the plane. T can be e.g. double or long long. (Avoid int.)

ef0c0e, 28L, 8m10s

```
template <class T> int sgn(T x) { return (x > 0) - (x < 0); }
template<class T>
struct Point {
  typedef Point P;
  T x, y;
  explicit Point(T _x=0, T _y=0) : x(_x), y(_y) {}
  bool operator<(P p) const { return tie(x,y) < tie(p.x,p.y); }
  bool operator==(P p) const { return tie(x,y)==tie(p.x,p.y); }
  P operator+(P p) const { return P(x+p.x, y+p.y); }
  P operator-(P p) const { return P(x-p.x, y-p.y); }
  P operator*(T d) const { return P(x*d, y*d); }
  P operator/(T d) const { return P(x/d, y/d); }
  T dot(P p) const { return x*p.x + y*p.y; }
  T cross(P p) const { return x*p.y - y*p.x; }
  T cross(P a, P b) const { return (a-*this).cross(b-*this); }
  T dist2() const { return x*x + y*y; }
  double dist() const { return sqrt((double)dist2()); }
  // angle to x-axis in interval [-pi, pi]
  double angle() const { return atan2(y, x); }
  P unit() const { return *this/dist(); } // makes dist()=1
  P perp() const { return P(-y, x); } // rotates +90 degrees
  P normal() const { return perp().unit(); }
  // returns point rotated 'a' radians ccw around the origin
  P rotate(double a) const {
    return P(x*cos(a)-y*sin(a),x*sin(a)+y*cos(a)); } // 4822a3
  friend ostream& operator<<(ostream& os, P p) {
    return os << "(" << p.x << "," << p.y << ")"; } // 9a9c95
}; // 6a5cce
```

lineDistance.h

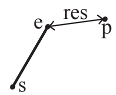
Description:
Returns the signed distance between point p and the line containing points a and b. Positive value on left side and negative on right as seen from a towards b. a==b gives nan. P is supposed to be Point<T> or Point3D<T> where T is e.g. double or long long. It uses products in intermediate steps so watch out for overflow if using int or long long. Using Point3D will always give a non-negative distance. For Point3D, call .dist on the result of the cross product.



"Point.h" f6bf6b, 4L, 50s

```
template<class P>
double lineDist(const P& a, const P& b, const P& p) {
  return (double)(b-a).cross(p-a)/(b-a).dist();
} // 00891c
```

SegmentDistance.h



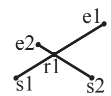
Description:
Returns the shortest distance between point p and the line segment from point s to e.
Usage: `Point<double> a, b(2,2), p(1,1);`
`bool onSegment = segDist(a,b,p) < 1e-10;`

"Point.h" 5c88f4, 6L, 1m20s

```
typedef Point<double> P;
double segDist(P& s, P& e, P& p) {
  if (s==e) return (p-s).dist();
  auto d = (e-s).dist2(), t = min(d,max(.0,(p-s).dot(e-s)));
  return ((p-s)*d-(e-s)*t).dist()/d;
} // ae751a
```

SegmentIntersection.h

Description:
If a unique intersection point between the line segments going from s1 to e1 and from s2 to e2 exists then it is returned. If no intersection point exists an empty vector is returned. If infinitely many exist a vector with 2 elements is returned, containing the endpoints of the common line segment. The wrong position will be returned if P is Point<ll> and the intersection point does not have integer coordinates. Products of three coordinates are used in intermediate steps so watch out for overflow if using int or long long.



Usage: `vector<P> inter = segInter(s1,e1,s2,e2);`
`if (sz(inter)==1)`
`cout << "segments intersect at " << inter[0] << endl;`

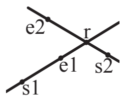
"Point.h", "OnSegment.h" 9d57f2, 13L, 3m10s

```
template<class P> vector<P> segInter(P a, P b, P c, P d) {
  auto oa = c.cross(d, a), ob = c.cross(d, b),
       oc = a.cross(b, c), od = a.cross(b, d);
  // Checks if intersection is single non-endpoint point.
  if (sgn(oa) * sgn(ob) < 0 && sgn(oc) * sgn(od) < 0)
```

```
    return {(a * ob - b * oa) / (ob - oa)};
  set<P> s;
  if (onSegment(c, d, a)) s.insert(a);
  if (onSegment(c, d, b)) s.insert(b);
  if (onSegment(a, b, c)) s.insert(c);
  if (onSegment(a, b, d)) s.insert(d);
  return {all(s)};
} // 9d57f2
```

lineIntersection.h

Description:
If a unique intersection point of the lines going through s1,e1 and s2,e2 exists {1, point} is returned. If no intersection point exists {0, (0,0)} is returned and if infinitely many exists {-1, (0,0)} is returned. The wrong position will be returned if P is Point<ll> and the intersection point does not have integer coordinates. Products of three coordinates are used in intermediate steps so watch out for overflow if using int or ll.



Usage: `auto res = lineInter(s1,e1,s2,e2);`
`if (res.first == 1)`
`cout << "intersection point at " << res.second << endl;`

"Point.h" a01f81, 8L, 1m50s

```
template<class P>
pair<int, P> lineInter(P s1, P e1, P s2, P e2) {
  auto d = (e1 - s1).cross(e2 - s2);
  if (d == 0) // if parallel
    return {-(s1.cross(e1, s2) == 0), P(0, 0)};
  auto p = s2.cross(e1, e2), q = s2.cross(e2, s1);
  return {1, (s1 * p + e1 * q) / d};
} // 47279a
```

sideOf.h

Description: Returns where p is as seen from s towards e. 1/0/-1 ⇔ left/on line/right. If the optional argument *eps* is given 0 is returned if p is within distance *eps* from the line. P is supposed to be Point<T> where T is e.g. double or long long. It uses products in intermediate steps so watch out for overflow if using int or long long.
Usage: `bool left = sideOf(p1,p2,q)==1;`

"Point.h" 3af81c, 9L, 1m40s

```
template<class P>
int sideOf(P s, P e, P p) { return sgn(s.cross(e, p)); }

template<class P>
int sideOf(const P& s, const P& e, const P& p, double eps) {
  auto a = (e-s).cross(p-s);
  double l = (e-s).dist()*eps;
  return (a > l) - (a < -l);
} // 33fa03
```

OnSegment.h

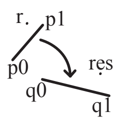
Description: Returns true iff p lies on the line segment from s to e. Use `(segDist(s,e,p)<=epsilon)` instead when using Point<double>.

"Point.h" c597e8, 3L, 40s

```
template<class P> bool onSegment(P s, P e, P p) {
  return p.cross(s, e) == 0 && (s - p).dot(e - p) <= 0;
} // c597e8
```

linearTransformation.h

Description:



Apply the linear transformation (translation, rotation and scaling) which takes line p0-p1 to line q0-q1 to point r.

"Point.h" 03a306, 6L, 1m40s

```
typedef Point<double> P;
P linearTransformation(const P& p0, const P& p1,
    const P& q0, const P& q1, const P& r) {
  P dp = p1-p0, dq = q1-q0, num(dp.cross(dq), dp.dot(dq));
  return q0 + P((r-p0).cross(num), (r-p0).dot(num))/dp.dist2();
} // 45ea01
```

LineProjectionReflection.h

Description: Projects point p onto line ab. Set refl=true to get reflection of point p across line ab instead. The wrong point will be returned if P is an integer point and the desired point doesn't have integer coordinates. Products of three coordinates are used in intermediate steps so watch out for overflow.

"Point.h" b5562d, 5L, 50s

```
template<class P>
P lineProj(P a, P b, P p, bool refl=false) {
  P v = b - a;
  return p - v.perp()*(1+refl)*v.cross(p-a)/v.dist2();
} // 4b716b
```

Angle.h

Description: A class for ordering angles (as represented by int points and a number of rotations around the origin). Useful for rotational sweeping. Sometimes also represents points or vectors.
Usage: `vector<Angle> v = {w[0], w[0].t360() ...}; // sorted`
`int j = 0; rep(i,0,n) { while (v[j] < v[i].t180()) ++j; }`
`sweeps j such that (j-i) represents the number of positively oriented triangles with vertices at 0 and i`

0f0602, 35L, 7m50s

```
struct Angle {
  int x, y;
  int t;
  Angle(int x, int y, int t=0) : x(x), y(y), t(t) {}
  Angle operator-(Angle b) const { return {x-b.x, y-b.y, t}; }
  int half() const {
    assert(x || y);
    return y < 0 || (y == 0 && x < 0);
  } // c935fb
  Angle t90() const { return {-y, x, t + (half() && x >= 0)}; }
  Angle t180() const { return {-x, -y, t + half()}; }
  Angle t360() const { return {x, y, t + 1}; }
}; // e258c0
bool operator<(Angle a, Angle b) {
  // add a.dist2() and b.dist2() to also compare distances
  return make_tuple(a.t, a.half(), a.y * (ll)b.x) <
         make_tuple(b.t, b.half(), a.x * (ll)b.y);
} // ce5ed3

// Given two points, this calculates the smallest angle between
// them, i.e., the angle that covers the defined line segment.
pair<Angle, Angle> segmentAngles(Angle a, Angle b) {
  if (b < a) swap(a, b);
  return (b < a.t180() ?
          make_pair(a, b) : make_pair(b, a.t360()));
} // 5eac29
Angle operator+(Angle a, Angle b) { // point a + vector b
  Angle r(a.x + b.x, a.y + b.y, a.t);
  if (a.t180() < r) r.t--;
  return r.t180() < a ? r.t360() : r;
} // 3d8073
Angle angleDiff(Angle a, Angle b) { // angle b - angle a
  int tu = b.t - a.t; a.t = b.t;
  return {a.x*b.x + a.y*b.y, a.x*b.y - a.y*b.x, tu - (b < a)};
} // ba3082
```

4.2 Circles

CircleIntersection.h

Description: Computes the pair of points at which two circles intersect. Returns false in case of no intersection.

"Point.h" 84d6d3, 11L, 2m50s

```
typedef Point<double> P;
bool circleInter(P a,P b,double r1,double r2,pair<P, P>* out) {
  if (a == b) { assert(r1 != r2); return false; }
  P vec = b - a;
  double d2 = vec.dist2(), sum = r1+r2, dif = r1-r2,
         p = (d2 + r1*r1 - r2*r2)/(d2*2), h2 = r1*r1 - p*p*d2;
  if (sum*sum < d2 || dif*dif > d2) return false;
  P mid = a + vec*p, per = vec.perp() * sqrt(fmax(0, h2) / d2);
  *out = {mid + per, mid - per};
  return true;
} // c64785
```

CircleTangents.h

Description: Finds the external tangents of two circles, or internal if r2 is negated. Can return 0, 1, or 2 tangents – 0 if one circle contains the other (or overlaps it, in the internal case, or if the circles are the same); 1 if the circles are tangent to each other (in which case .first = .second and the tangent line is perpendicular to the line between the centers). .first and .second give the tangency points at circle 1 and 2 respectively. To find the tangents of a circle with a point set r2 to 0.

"Point.h" b0153d, 13L, 2m40s

```
template<class P>
vector<pair<P, P>> tangents(P c1, double r1, P c2, double r2) {
  P d = c2 - c1;
  double dr = r1 - r2, d2 = d.dist2(), h2 = d2 - dr * dr;
  if (d2 == 0 || h2 < 0)  return {};
  vector<pair<P, P>> out;
  for (double sign : {-1, 1}) {
    P v = (d * dr + d.perp() * sqrt(h2) * sign) / d2;
    out.push_back({c1 + v * r1, c2 + v * r2});
  } // e25263
  if (h2 == 0) out.pop_back();
  return out;
} // 4835b9
```

CircleLine.h

Description: Finds the intersection between a circle and a line. Returns a vector of either 0, 1, or 2 intersection points. P is intended to be Point<double>.

"Point.h" e0cfba, 9L, 1m50s

```
template<class P>
vector<P> circleLine(P c, double r, P a, P b) {
  P ab = b - a, p = a + ab * (c-a).dot(ab) / ab.dist2();
  double s = a.cross(b, c), h2 = r*r - s*s / ab.dist2();
  if (h2 < 0) return {};
  if (h2 == 0) return {p};
  P h = ab.unit() * sqrt(h2);
  return {p - h, p + h};
} // 59aded
```

CirclePolygonIntersection.h

Description: Returns the area of the intersection of a circle with a ccw polygon.
Time: $\mathcal{O}(n)$

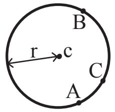
"../../content/geometry/Point.h" 19add1, 19L, 4m10s

```
typedef Point<double> P;
#define arg(p, q) atan2(p.cross(q), p.dot(q))
double circlePoly(P c, double r, vector<P> ps) {
  auto tri = [&](P p, P q) {
    auto r2 = r * r / 2;
    P d = q - p;
    auto a = d.dot(p)/d.dist2(), b = (p.dist2()-r*r)/d.dist2();
    auto det = a * a - b;
    if (det <= 0) return arg(p, q) * r2;
    auto s = max(0., -a-sqrt(det)), t = min(1., -a+sqrt(det));
    if (t < 0 || 1 <= s) return arg(p, q) * r2;
```

```
    P u = p + d * s, v = q + d * (t-1);
    return arg(p,u) * r2 + u.cross(v)/2 + arg(v,q) * r2;
  }; // a526fe
  auto sum = 0.0;
  rep(i,0,sz(ps))
    sum += tri(ps[i] - c, ps[(i + 1) % sz(ps)] - c);
  return sum;
} // f082e0
```

circumcircle.h

Description:



The circumcirle of a triangle is the circle intersecting all three vertices. ccRadius returns the radius of the circle going through points A, B and C and ccCenter returns the center of the same circle.

"Point.h" 1caa3a, 9L, 1m50s

```
typedef Point<double> P;
double ccRadius(const P& A, const P& B, const P& C) {
  return (B-A).dist()*(C-B).dist()*(A-C).dist()/
      abs((B-A).cross(C-A))/2;
} // 607d98
P ccCenter(const P& A, const P& B, const P& C) {
  P b = C-A, c = B-A;
  return A + (b*c.dist2()-c*b.dist2()).perp()/b.cross(c)/2;
} // 79372e
```

MinimumEnclosingCircle.h

Description: Computes the minimum circle that encloses a set of points.
Time: expected $\mathcal{O}(n)$

"circumcircle.h" 09dd0a, 17L, 3m

```
pair<P, double> mec(vector<P> ps) {
  shuffle(all(ps), mt19937(time(0)));
  P o = ps[0];
  double r = 0, EPS = 1 + 1e-8;
  rep(i,0,sz(ps)) if ((o - ps[i]).dist() > r * EPS) {
    o = ps[i], r = 0;
    rep(j,0,i) if ((o - ps[j]).dist() > r * EPS) {
      o = (ps[i] + ps[j]) / 2;
      r = (o - ps[i]).dist();
      rep(k,0,j) if ((o - ps[k]).dist() > r * EPS) {
        o = ccCenter(ps[i], ps[j], ps[k]);
        r = (o - ps[i]).dist();
      } // 64802f
    } // 7b0ecf
  } // dcfd0e
  return {o, r};
} // 09dd0a
```

4.3 Polygons

InsidePolygon.h

Description: Returns true if p lies within the polygon. If strict is true, it returns false for points on the boundary. The algorithm uses products in intermediate steps so watch out for overflow.
Usage: vector<P> v = {P{4,4}, P{1,2}, P{2,1}};
bool in = inPolygon(v, P{3, 3}, false);
Time: $\mathcal{O}(n)$

"Point.h", "OnSegment.h", "SegmentDistance.h" 2bf504, 11L, 2m10s

```
template<class P>
bool inPolygon(vector<P> &p, P a, bool strict = true) {
  int cnt = 0, n = sz(p);
  rep(i,0,n) {
    P q = p[(i + 1) % n];
    if (onSegment(p[i], q, a)) return !strict;
    //or: if (segDist(p[i], q, a) <= eps) return !strict;
    cnt ^= ((a.y<p[i].y) - (a.y<q.y)) * a.cross(p[i], q) > 0;
  } // 1b9961
  return cnt;
} // c7225e
```

PolygonCenter.h

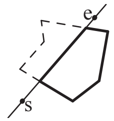
Description: Returns the center of mass for a polygon.
Time: $\mathcal{O}(n)$

"Point.h" 9706dc, 9L, 1m40s

```
typedef Point<double> P;
P polygonCenter(const vector<P>& v) {
  P res(0, 0); double A = 0;
  for (int i = 0, j = sz(v) - 1; i < sz(v); j = i++) {
    res = res + (v[i] + v[j]) * v[j].cross(v[i]);
    A += v[j].cross(v[i]);
  } // 307102
  return res / A / 3;
} // 0d0d84
```

PolygonCut.h

Description:



Returns a vector with the vertices of a polygon with everything to the left of the line going from s to e cut away.
Usage: vector<P> p = ...;
p = polygonCut(p, P(0,0), P(1,0));

"Point.h" d07181, 13L, 2m20s

```
typedef Point<double> P;
vector<P> polygonCut(const vector<P>& poly, P s, P e) {
  vector<P> res;
  rep(i,0,sz(poly)) {
    P cur = poly[i], prev = i ? poly[i-1] : poly.back();
    auto a = s.cross(e, cur), b = s.cross(e, prev);
    if ((a < 0) != (b < 0))
      res.push_back(cur + (prev - cur) * (a / (a - b)));
    if (a < 0)
      res.push_back(cur);
  } // 757c0d
  return res;
} // 42c993
```

PolygonUnion.h

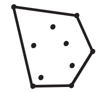
Description: Calculates the area of the union of n polygons (not necessarily convex). The points within each polygon must be given in CCW order. (Epsilon checks may optionally be added to sideOf/sgn, but shouldn't be needed.)
Time: $\mathcal{O}\left(N^2\right)$, where N is the total number of points

"Point.h", "sideOf.h" 3931c6, 33L, 7m10s

```
typedef Point<double> P;
double rat(P a, P b) { return sgn(b.x) ? a.x/b.x : a.y/b.y; }
double polyUnion(vector<vector<P>>& poly) {
  double ret = 0;
  rep(i,0,sz(poly)) rep(v,0,sz(poly[i])) {
    P A = poly[i][v], B = poly[i][(v + 1) % sz(poly[i])];
    vector<pair<double, int>> segs = {{0, 0}, {1, 0}};
    rep(j,0,sz(poly)) if (i != j) {
      rep(u,0,sz(poly[j])) {
        P C = poly[j][u], D = poly[j][(u + 1) % sz(poly[j])];
        int sc = sideOf(A, B, C), sd = sideOf(A, B, D);
        if (sc != sd) {
          double sa = C.cross(D, A), sb = C.cross(D, B);
          if (min(sc, sd) < 0)
            segs.emplace_back(sa / (sa - sb), sgn(sc - sd));
        } else if (!sc && !sd && j<i && sgn((B-A).dot(D-C))>0){
            // cf73dd
          segs.emplace_back(rat(C - A, B - A), 1);
          segs.emplace_back(rat(D - A, B - A), -1);
        } // 313ead
      } // 0d1e2c
    } // fdc855
    sort(all(segs));
    for (auto& s : segs) s.first = min(max(s.first, 0.0), 1.0);
    double sum = 0;
    int cnt = segs[0].second;
    rep(j,1,sz(segs)) {
      if (!cnt) sum += segs[j].first - segs[j - 1].first;
      cnt += segs[j].second;
    } // f5884f
    ret += A.cross(B) * sum;
  } // 19180c
  return ret / 2;
} // 6e88cf
```

ConvexHull.h

Description:



Returns a vector of the points of the convex hull in counterclockwise order. Points on the edge of the hull between two other points are not considered part of the hull. To include these "borders", follow the code comments.
Time: $\mathcal{O}(n \log n)$

"Point.h" 310954, 13L, 2m50s

```
typedef Point<ll> P;
vector<P> convexHull(vector<P> pts) {
  if (sz(pts) <= 1) return pts;
  sort(all(pts));
  vector<P> h(sz(pts)+1); // 2*sz(pts) to include borders
  int s = 0, t = 0;
  for (int it = 2; it--; s = --t, reverse(all(pts)))
    for (P p : pts) {
      while (t >= s + 2 && h[t-2].cross(h[t-1], p) <= 0) t--;
      h[t++] = p; // < above to include borders
    } // bf0344
  return {h.begin(), h.begin() + t - (t == 2 && h[0] == h[1])};
} // ec85f8
```

HullDiameter.h

Description: Returns the two points with max distance on a convex hull (ccw, no duplicate/collinear points).
Time: $\mathcal{O}(n)$

"Point.h" c571b8, 12L, 2m20s

```
typedef Point<ll> P;
array<P, 2> hullDiameter(vector<P> S) {
  int n = sz(S), j = n < 2 ? 0 : 1;
  pair<ll, array<P, 2>> res({0, {S[0], S[0]}});
  rep(i,0,j)
    for (;; j = (j + 1) % n) {
      res = max(res, {(S[i] - S[j]).dist2(), {S[i], S[j]}});
      if ((S[(j + 1) % n] - S[j]).cross(S[i + 1] - S[i]) >= 0)
        break;
    } // 56cc40
  return res.second;
} // 5f726b
```

PointInsideHull.h

Description: Determine whether a point t lies inside a convex hull (CCW order, with no collinear points). Returns true if point lies within the hull. If strict is true, points on the boundary aren't included.
Time: $\mathcal{O}(\log N)$

"Point.h", "sideOf.h", "OnSegment.h" 71446b, 14L, 3m

```
typedef Point<ll> P;

bool inHull(const vector<P>& l, P p, bool strict = true) {
  int a = 1, b = sz(l) - 1, r = !strict;
  if (sz(l) < 3) return r && onSegment(l[0], l.back(), p);
```

```
  if (sideOf(l[0], l[a], l[b]) > 0) swap(a, b);
  if (sideOf(l[0], l[a], p) >= r || sideOf(l[0], l[b], p)<= -r)
    return false;
  while (abs(a - b) > 1) {
    int c = (a + b) / 2;
    (sideOf(l[0], l[c], p) > 0 ? b : a) = c;
  } // b265ab
  return sgn(l[a].cross(l[b], p)) < r;
} // c74639
```

LineHullIntersection.h

Description: Line-convex polygon intersection. The polygon must be ccw and have no collinear points. lineHull(line, poly) returns a pair describing the intersection of a line with the polygon: • $(-1,-1)$ if no collision, • $(i,-1)$ if touching the corner i, • (i,i) if along side $(i,i+1)$, • (i,j) if crossing sides $(i,i+1)$ and $(j,j+1)$. In the last case, if a corner i is crossed, this is treated as happening on side $(i,i+1)$. The points are returned in the same order as the line hits the polygon. `extrVertex` returns the point of a hull with the max projection onto a line.
Time: $\mathcal{O}(\log n)$

"Point.h" 7cf45b, 38L, 8m

```
#define cmp(i,j) sgn(dir.perp().cross(poly[(i)%n]-poly[(j)%n]))
#define extr(i) cmp(i + 1, i) >= 0 && cmp(i, i - 1 + n) < 0
template <class P> int extrVertex(vector<P>& poly, P dir) {
  int n = sz(poly), lo = 0, hi = n;
  if (extr(0)) return 0;
  while (lo + 1 < hi) {
    int m = (lo + hi) / 2;
    if (extr(m)) return m;
    int ls = cmp(lo + 1, lo), ms = cmp(m + 1, m);
    (ls < ms || (ls == ms && ls == cmp(lo, m)) ? hi : lo) = m;
  } // 68a24c
  return lo;
} // 7f0477
#define cmpL(i) sgn(a.cross(poly[i], b))
template <class P>
array<int, 2> lineHull(P a, P b, vector<P>& poly) {
  int endA = extrVertex(poly, (a - b).perp());
  int endB = extrVertex(poly, (b - a).perp());
  if (cmpL(endA) < 0 || cmpL(endB) > 0)
    return {-1, -1};
  array<int, 2> res;
  rep(i,0,2) {
    int lo = endB, hi = endA, n = sz(poly);
    while ((lo + 1) % n != hi) {
      int m = ((lo + hi + (lo < hi ? 0 : n)) / 2) % n;
      (cmpL(m) == cmpL(endB) ? lo : hi) = m;
    } // 52528c
    res[i] = (lo + !cmpL(hi)) % n;
    swap(endA, endB);
  } // c05c70
  if (res[0] == res[1]) return {res[0], -1};
  if (!cmpL(res[0]) && !cmpL(res[1]))
    switch ((res[0] - res[1] + sz(poly) + 1) % sz(poly)) {
      case 0: return {res[0], res[0]};
      case 2: return {res[1], res[1]};
    } // 8fa383
  return res;
} // 36fc8e
```

PolygonContainmentTree.h

Description: building tree of polygon containment
Memory: $\mathcal{O}(N)$
Time: $\mathcal{O}(N \log N)$

14c82a, 44L, 8m40s

```
struct P { ll x, y; };

int current_x;
struct Segment {
  int idx; P p1, p2; bool is_upper;
  Segment(P p, P q, int i): idx(i), p1(p), p2(q), is_upper(p2.x
      < p1.x) { if (is_upper)swap(p1, p2); }
  ld get_y(ll x) const { return (ld) (p2.y - p1.y) / (p2.x - p1
      .x) * (x - p1.x) + p1.y; }
  tuple<ld, bool, int> get_comp() const { return {get_y(
      current_x), is_upper, p2.x}; }
  bool operator<(const Segment & o) const { return get_comp() <
      o.get_comp(); }
}; // fc8b4f

vector<int> build(vector<vector<P>>& polygons) {
  int n = sz(polygons);
  vector<tuple<int, int, int, Segment>> edges; // polygon edges
  rep(idx, 0, n) {
    const auto & v = polygons[idx];
    rep(i, 0, sz(v)) {
      int j = (i + 1) % sz(v);
      if (v[i].x == v[j].x)continue; // ignores vertical edges
      Segment seg = Segment(v[i], v[j], idx);
      edges.eb(seg.p1.x, 0, -seg.p1.y, seg);
      edges.eb(seg.p2.x, 1, -seg.p2.y, seg);
    } // b28b76
  } // 2603da
  sort(edges.begin(), edges.end());
  set&lt;Segment&gt; s;
  vector pai(n+1, n), vis(n, 0);
  for (auto [l, t, y, seg]: edges) {
    current_x = l;
    int i = seg.idx;
    if (t == 0) {
      if (not vis[i]) {
        vis[i] = true;
        auto it = s.upper_bound(seg);
        if (it == s.end())pai[i] = n;
        else if (it->is_upper)pai[i] = it->idx;
        else pai[i] = pai[it->idx];
      } // 17d107
      s.insert(seg);
    } // aff5b0
    else s.erase(seg);
  } // 3514ff
  return pai;
} // 0b7cbd
```

Minkowski.h

Description: Calculates the Minkowski sum of two Polygons in ccw order
Time: $\mathcal{O}(N + M)$

"Point.h" e510cd, 19L, 4m10s

```
template<typename P>
vector<P> minkowski(vector<P> p, vector<P> q){
  auto comp = [](auto a, auto b) {
      return make_pair(a.y, a.x) < make_pair(b.y, b.x);
  }; // 278e98
    rotate(p.begin(), min_element(all(p), comp), p.end());
    rotate(q.begin(), min_element(all(q), comp), q.end());
    p.pb(p[0]); p.pb(p[1]);
    q.pb(q[0]); q.pb(q[1]);
    vector<P> result;
    int i = 0, j = 0;
    while(i < sz(p) - 2 || j < sz(q) - 2){
        result.pb(p[i] + q[j]);
        auto cross = (p[i + 1] - p[i]).cross(q[j + 1] - q[j]);
        if(cross >= 0 && i < sz(p) - 2) i++;
        if(cross <= 0 && j < sz(q) - 2) j++;
    } // 57f2d7
    return result;
} // a99371
```

HalfPlane.h

Description: Calculates the halfplane intersection of Halfplanes, Halfplanes allow everything to the left. Cuidado com a precisao
Time: $O(N \log(N)$

"Point.h" 9cf69d, 50L, 10m40s

```
const ld EPS = 1e-9, INF = 1e9;
typedef Point<ld> P;
struct HP {
    P p, pq;
  ld angle;
    HP() {}
    HP(P a, P b) : p(a), pq(b - a), angle(atan2(pq.y, pq.x)) {}

    bool out(const P & r) { return pq.cross(r - p) < -EPS; }
    bool operator<(const HP & e) const { return angle < e.angle
        ; }
    friend P inter(const HP & s, const HP & t) {
        ld alpha = (t.p - s.p).cross(t.pq) / s.pq.cross(t.pq);
        return s.p + (s.pq * alpha);
    } // 946b1a
}; // 782415
vector<P> hp_intersect(vector<HP> &H) {
    P box[4] = {
        P(INF, INF),
        P(-INF, INF),
        P(-INF, -INF),
        P(INF, -INF)
    }; // 63b3ab
    for(int i = 0; i < 4; i++) H.eb(box[i], box[(i+1) % 4]);
    sort(H.begin(), H.end());
    deque<HP> dq;
    int len = 0;
    rep(i, 0, sz(H)) {
        while (len > 1 && H[i].out(inter(dq[len-1], dq[len-2]))
            )
            dq.pop_back(), len--;
        while (len > 1 && H[i].out(inter(dq[0], dq[1])))
            dq.pop_front(), len--;
    // Special case check: Parallel half planes
        if (len > 0 && fabsl(H[i].pq.cross(dq[len-1].pq)) < EPS
            ) {
            if (H[i].pq.dot(dq[len-1].pq) < 0.0) return {};
            if (H[i].out(dq[len-1].p))
                dq.pop_back(), len--;
            else continue;
        } // 6ef632
        dq.push_back(H[i]); len++;
    } // 82e81e
    while (len > 2 && dq[0].out(inter(dq[len-1], dq[len-2])))
        dq.pop_back(), len--;
    while (len > 2 && dq[len-1].out(inter(dq[0], dq[1])))
    dq.pop_front(), len--;
    if (len < 3) return {};
    vector<P> ret(len);
  rep(i, 0, len-1) ret[i] = inter(dq[i], dq[i+1]);
    ret.back() = inter(dq[len-1], dq[0]);
    return ret;
} // 586596
```

4.4 Misc. Point Set Problems

ClosestPair.h

Description: Finds the closest pair of points.
Time: $\mathcal{O}(n \log n)$

"Point.h" ac41a6, 17L, 3m10s

```
typedef Point<ll> P;
```

```
pair<P, P> closest(vector<P> v) {
  assert(sz(v) > 1);
  set<P> S;
  sort(all(v), [](P a, P b) { return a.y < b.y; });
  pair<ll, pair<P, P>> ret{LLONG_MAX, {P(), P()}};
  int j = 0;
  for (P p : v) {
    P d{1 + (ll)sqrt(ret.first), 0};
    while (v[j].y <= p.y - d.x) S.erase(v[j++]);
    auto lo = S.lower_bound(p - d), hi = S.upper_bound(p + d);
    for (; lo != hi; ++lo)
      ret = min(ret, {(*lo - p).dist2(), {*lo, p}});
    S.insert(p);
  } // 5b096c
  return ret.second;
} // bf22c6
```

ManhattanMST.h

Description: Given N points, returns up to 4*N edges, which are guaranteed to contain a minimum spanning tree for the graph with edge weights w(p, q) = —p.x - q.x— + —p.y - q.y—. Edges are in the form (distance, src, dst). Use a standard MST algorithm on the result to find the final MST.
Time: $\mathcal{O}(N \log N)$

"Point.h" df6f59, 23L, 4m

```
typedef Point<int> P;
vector<array<int, 3>> manhattanMST(vector<P> ps) {
  vi id(sz(ps));
  iota(all(id), 0);
  vector<array<int, 3>> edges;
  rep(k,0,4) {
    sort(all(id), [&](int i, int j) {
         return (ps[i]-ps[j]).x < (ps[j]-ps[i]).y;}); // 02bd87
    map<int, int> sweep;
    for (int i : id) {
      for (auto it = sweep.lower_bound(-ps[i].y);
                it != sweep.end(); sweep.erase(it++)) {
        int j = it->second;
        P d = ps[i] - ps[j];
        if (d.y > d.x) break;
        edges.push_back({d.y + d.x, i, j});
      } // 271a42
      sweep[-ps[i].y] = i;
    } // e692f6
    for (P& p : ps) if (k & 1) p.x = -p.x; else swap(p.x, p.y);
  } // a110e0
  return edges;
} // a1195f
```

FastDelaunay.h

Description: Fast Delaunay triangulation. Each circumcircle contains none of the input points. There must be no duplicate points. If all points are on a line, no triangles will be returned. Should work for doubles as well, though there may be precision issues in 'circ'. Returns triangles in order {t[0][0], t[0][1], t[0][2], t[1][0], . . .}, all counter-clockwise.
Time: $\mathcal{O}(n \log n)$

"Point.h" eefdf5, 82L, 17m10s

```
typedef Point<ll> P;
typedef struct Quad* Q;
typedef __int128_t lll; // (can be ll if coords are < 2e4)
P arb(LLONG_MAX,LLONG_MAX); // not equal to any other point
struct Quad {
  Q rot, o; P p = arb; bool mark;
  P& F() { return r()->p; }
  Q& r() { return rot->rot; }
  Q prev() { return rot->o->rot; }
  Q next() { return r()->prev(); }
} *H; // 18059e
bool circ(P p, P a, P b, P c) { // is p in the circumcircle?
  lll p2 = p.dist2(), A = a.dist2()-p2,
      B = b.dist2()-p2, C = c.dist2()-p2;
  return p.cross(a,b)*C + p.cross(b,c)*A + p.cross(c,a)*B > 0;
} // 6aff7b
Q makeEdge(P orig, P dest) {
  Q r = H ? H : new Quad{new Quad{new Quad{new Quad{0}}}};
  H = r->o; r->r()->r() = r;
  rep(i,0,4) r = r->rot, r->p = arb, r->o = i & 1 ? r : r->r();
  r->p = orig; r->F() = dest;
  return r;
} // b3b5b1
void splice(Q a, Q b) {
  swap(a->o->rot->o, b->o->rot->o); swap(a->o, b->o);
} // 86ce01
Q connect(Q a, Q b) {
  Q q = makeEdge(a->F(), b->p);
  splice(q, a->next());
  splice(q->r(), b);
  return q;
} // 4a4fc2
pair<Q,Q> rec(const vector<P>& s) {
  if (sz(s) <= 3) {
    Q a = makeEdge(s[0], s[1]), b = makeEdge(s[1], s.back());
    if (sz(s) == 2) return { a, a->r() };
    splice(a->r(), b);
    auto side = s[0].cross(s[1], s[2]);
    Q c = side ? connect(b, a) : 0;
    return {side < 0 ? c->r() : a, side < 0 ? c : b->r() };
  } // c9e598
#define H(e) e->F(), e->p
#define valid(e) (e->F().cross(H(base)) > 0)
  Q A, B, ra, rb;
  int half = sz(s) / 2;
  tie(ra, A) = rec({all(s) - half});
  tie(B, rb) = rec({sz(s) - half + all(s)});
  while ((B->p.cross(H(A)) < 0 && (A = A->next())) ||
         (A->p.cross(H(B)) > 0 && (B = B->r()->o)));
  Q base = connect(B->r(), A);
  if (A->p == ra->p) ra = base->r();
  if (B->p == rb->p) rb = base;
#define DEL(e, init, dir) Q e = init->dir; if (valid(e)) \
    while (circ(e->dir->F(), H(base), e->F())) { \
      Q t = e->dir; \
      splice(e, e->prev()); \
      splice(e->r(), e->r()->prev()); \
      e->o = H; H = e; e = t; \
    } // a2e9b5
  for (;;) {
    DEL(LC, base->r(), o);  DEL(RC, base, prev());
    if (!valid(LC) && !valid(RC)) break;
    if (!valid(LC) || (valid(RC) && circ(H(RC), H(LC))))
      base = connect(RC, base->r());
    else
      base = connect(base->r(), LC->r());
  } // fcf7ef
  return { ra, rb };
} // 7cf639
vector<P> triangulate(vector<P> pts) {
  sort(all(pts));  assert(unique(all(pts)) == pts.end());
  if (sz(pts) < 2) return {};
  Q e = rec(pts).first;
  vector<Q> q = {e};
  int qi = 0;
  while (e->o->F().cross(e->F(), e->p) < 0) e = e->o;
#define ADD { Q c = e; do { c->mark = 1; pts.push_back(c->p); \
  q.push_back(c->r()); c = c->next(); } while (c != e); } // 43
      e195 // 43e195
  ADD; pts.clear();
  while (qi < sz(q)) if (!(e = q[qi++])->mark) ADD;
  return pts;
} // a02307
```

4.5 3D

PolyhedronVolume.h

Description: Magic formula for the volume of a polyhedron. Faces should point outwards.

3058c3, 6L, 1m10s

```
template<class V, class L>
double signedPolyVolume(const V& p, const L& trilist) {
  double v = 0;
  for (auto i : trilist) v += p[i.a].cross(p[i.b]).dot(p[i.c]);
  return v / 6;
} // fca9df
```

Point3D.h

Description: Class to handle points in 3D space. T can be e.g. double or long long.

8058ae, 32L, 9m

```
template<class T> struct Point3D {
  typedef Point3D P;
  typedef const P& R;
  T x, y, z;
  explicit Point3D(T x=0, T y=0, T z=0) : x(x), y(y), z(z) {}
  bool operator<(R p) const {
    return tie(x, y, z) < tie(p.x, p.y, p.z); } // 8eef6b
  bool operator==(R p) const {
    return tie(x, y, z) == tie(p.x, p.y, p.z); } // bd6a08
  P operator+(R p) const { return P(x+p.x, y+p.y, z+p.z); }
  P operator-(R p) const { return P(x-p.x, y-p.y, z-p.z); }
  P operator*(T d) const { return P(x*d, y*d, z*d); }
  P operator/(T d) const { return P(x/d, y/d, z/d); }
  T dot(R p) const { return x*p.x + y*p.y + z*p.z; }
  P cross(R p) const {
    return P(y*p.z - z*p.y, z*p.x - x*p.z, x*p.y - y*p.x);
  } // a77b7e
  T dist2() const { return x*x + y*y + z*z; }
  double dist() const { return sqrt((double)dist2()); }
  //Azimuthal angle (longitude) to x-axis in interval [-pi, pi]
  double phi() const { return atan2(y, x); }
  //Zenith angle (latitude) to the z-axis in interval [0, pi]
  double theta() const { return atan2(sqrt(x*x+y*y),z); }
  P unit() const { return *this/(T)dist(); } //makes dist()=1
  //returns unit vector normal to *this and p
  P normal(P p) const { return cross(p).unit(); }
  //returns point rotated 'angle' radians ccw around axis
  P rotate(double angle, P axis) const {
    double s = sin(angle), c = cos(angle); P u = axis.unit();
    return u*dot(u)*(1-c) + (*this)*c - cross(u)*s;
  } // 73af70
}; // 8058ae
```

3dHull.h

Description: Computes all faces of the 3-dimension hull of a point set. *No four points must be coplanar*, or else random results will be returned. All faces will point outwards.
Time: $\mathcal{O}(n^2)$

"Point3D.h" 5b45fc, 46L, 7m50s

```
typedef Point3D<double> P3;
struct PR {
  void ins(int x) { (a == -1 ? a : b) = x; }
  void rem(int x) { (a == x ? a : b) = -1; }
  int cnt() { return (a != -1) + (b != -1); }
  int a, b;
}; // cf7c9e
struct F { P3 q; int a, b, c; };
```

```
vector<F> hull3d(const vector<P3>& A) {
  assert(sz(A) >= 4);
  vector<vector<PR>> E(sz(A), vector<PR>(sz(A), {-1, -1}));
#define E(x,y) E[f.x][f.y]
  vector<F> FS;
  auto mf = [&](int i, int j, int k, int l) {
    P3 q = (A[j] - A[i]).cross((A[k] - A[i]));
    if (q.dot(A[l]) > q.dot(A[i]))
      q = q * -1;
    F f{q, i, j, k};
    E(a,b).ins(k); E(a,c).ins(j); E(b,c).ins(i);
    FS.push_back(f);
  }; // d73a06
  rep(i,0,4) rep(j,i+1,4) rep(k,j+1,4)
    mf(i, j, k, 6 - i - j - k);

  rep(i,4,sz(A)) {
    rep(j,0,sz(FS)) {
      F f = FS[j];
      if(f.q.dot(A[i]) > f.q.dot(A[f.a])) {
        E(a,b).rem(f.c);
        E(a,c).rem(f.b);
        E(b,c).rem(f.a);
        swap(FS[j--], FS.back());
        FS.pop_back();
      } // 5cd5dc
    } // 220067
    int nw = sz(FS);
    rep(j,0,nw) {
      F f = FS[j];
#define C(a, b, c) if (E(a,b).cnt() != 2) mf(f.a, f.b, i, f.c);
      C(a, b, c); C(a, c, b); C(b, c, a);
    } // 248ed4
  } // 47289c
  for (F& it : FS) if ((A[it.b] - A[it.a]).cross(
    A[it.c] - A[it.a]).dot(it.q) <= 0) swap(it.c, it.b);
  return FS;
}; // be2ca2
```

sphericalDistance.h

Description: Returns the shortest distance on the sphere with radius radius between the points with azimuthal angles (longitude) f1 (ϕ_1) and f2 (ϕ_2) from x axis and zenith angles (latitude) t1 (θ_1) and t2 (θ_2) from z axis (0 = north pole). All angles measured in radians. The algorithm starts by converting the spherical coordinates to cartesian coordinates so if that is what you have you can use only the two last rows. dx*radius is then the difference between the two points in the x direction and d*radius is the total distance between the points.

611f07, 8L, 1m50s

```
double sphericalDistance(double f1, double t1,
    double f2, double t2, double radius) {
  double dx = sin(t2)*cos(f2) - sin(t1)*cos(f1);
  double dy = sin(t2)*sin(f2) - sin(t1)*sin(f1);
  double dz = cos(t2) - cos(t1);
  double d = sqrt(dx*dx + dy*dy + dz*dz);
  return radius*2*asin(d/2);
} // 4fa19e
```

Graph (5)

5.1 Fundamentals

BellmanFord.h

Description: Calculates shortest paths from s in a graph that might have negative edge weights. Unreachable nodes get dist = inf; nodes reachable through negative-weight cycles get dist = -inf. Assumes $V^2 \max |w_i| < \sim 2^{63}$.
Time: $\mathcal{O}(VE)$

830a8f, 21L, 4m30s

```
const ll inf = LLONG_MAX;
struct Ed { int a, b, w, s() { return a < b ? a : -a; }};
struct Node { ll dist = inf; int prev = -1; };
void bellmanFord(vector<Node>& nodes, vector<Ed>& eds, int s) {
  nodes[s].dist = 0;
  sort(all(eds), [](Ed a, Ed b) { return a.s() < b.s(); });
  int lim = sz(nodes) / 2 + 2; // /3+100 with shuffled vertices
  rep(i,0,lim) for (Ed ed : eds) {
    Node cur = nodes[ed.a], &dest = nodes[ed.b];
    if (abs(cur.dist) == inf) continue;
    ll d = cur.dist + ed.w;
    if (d < dest.dist) {
      dest.prev = ed.a;
      dest.dist = (i < lim-1 ? d : -inf);
    } // 452019
  } // 75a370
  rep(i,0,lim) for (Ed e : eds) {
    if (nodes[e.a].dist == -inf)
      nodes[e.b].dist = -inf;
  } // 1d7315
} // fa39de
```

FunctGraph.h

Description: Functional Graph
Memory: $\mathcal{O}(n)$
Time: $\mathcal{O}(n)$

152fc5, 24L, 4m

```
struct FunctGraph{
  int n;
  vi head, comp;
  vector<vi> gr, cycles;
  FunctGraph(vi& fn):
    n(sz(fn)), head(n, -1), comp(n), gr(n) {
    rep(i, 0, n)gr[fn[i]].pb(i);
    vi visited(n, 0);
    auto dfs = [&](auto rec, int v, int c) -> void {
      head[v] = c; visited[v] = 1;
      for(int f : gr[v])if (head[f]!=f)rec(rec, f, c);
    }; // e1fa06
    rep(i, 0, n){
      if (visited[i])continue;
      int l=fn[i], r=fn[fn[i]];
      while(l!=r) l=fn[l], r=fn[fn[r]];
      vi cur = {r};
      for(l=fn[l]; l!=r; l=fn[l]) cur.pb(l);
      for(int x : cur) head[x] = x, comp[x] = sz(cycles);
      cycles.pb(cur);
      for(int x : cur) dfs(dfs, x, x);
    } // 01a153
  } // 0a2937
}; // 152fc5
```

5.2 Flow

Dinic.h

Description: Flow algorithm with complexity $O(VE\log U)$ where $U = \max|\text{cap}|$. $O(\min(E^{1/2}, V^{2/3})E)$ if $U = 1$; $O(\sqrt{V}E)$ for bipartite matching.

d7f0f1, 42L, 7m30s

```
struct Dinic {
  struct Edge {
    int to, rev;
    ll c, oc;
    ll flow() { return max(oc - c, 0LL); } // if you need flows
  }; // d46531
  vi lvl, ptr, q;
  vector<vector<Edge>> adj;
  Dinic(int n) : lvl(n), ptr(n), q(n), adj(n) {}
  void addEdge(int a, int b, ll c, ll rcap = 0) {
    adj[a].push_back({b, sz(adj[b]), c, c});
    adj[b].push_back({a, sz(adj[a]) - 1, rcap, rcap});
  } // 01ad91
  ll dfs(int v, int t, ll f) {
    if (v == t || !f) return f;
    for (int& i = ptr[v]; i < sz(adj[v]); i++) {
      Edge& e = adj[v][i];
      if (lvl[e.to] == lvl[v] + 1)
        if (ll p = dfs(e.to, t, min(f, e.c))) {
          e.c -= p, adj[e.to][e.rev].c += p;
          return p;
        } // 494522
    } // f73941
    return 0;
  } // 0922b8
  ll calc(int s, int t) {
    ll flow = 0; q[0] = s;
    rep(L,0,31) do { // 'int L=30' maybe faster for random data
      lvl = ptr = vi(sz(q));
      int qi = 0, qe = lvl[s] = 1;
      while (qi < qe && !lvl[t]) {
        int v = q[qi++];
        for (Edge e : adj[v])
          if (!lvl[e.to] && e.c >> (30 - L))
            q[qe++] = e.to, lvl[e.to] = lvl[v] + 1;
      } // 80ef52
      while (ll p = dfs(s, t, LLONG_MAX)) flow += p;
    } while (lvl[t]); // 96458d
    return flow;
  } // 3eb6d1
  bool leftOfMinCut(int a) { return lvl[a] != 0; }
}; // d7f0f1
```

MinCostMaxFlow.h

Description: Min-cost max-flow. If costs can be negative, call setpi before maxflow, but note that negative cost cycles are not supported. To obtain the actual flow, look at positive values only.
Time: $\mathcal{O}(FE\log(V))$ where F is max flow. $\mathcal{O}(VE)$ for setpi.

<bits/extc++.h> 135b73, 69L, 13m

```
const ll INF = numeric_limits<ll>::max() / 4;
struct MCMF {
  struct edge {
    int from, to, rev;
    ll cap, cost, flow;
  }; // 092ff8
  int N;
  vector<vector<edge>> ed;
  vi seen;
  vector<ll> dist, pi;
  vector<edge*> par;
  MCMF(int N) : N(N), ed(N), seen(N), dist(N), pi(N), par(N) {}
  void addEdge(int from, int to, ll cap, ll cost) {
    if (from == to) return;
    ed[from].push_back(edge{ from,to,sz(ed[to]),cap,cost,0 });
    ed[to].push_back(edge{ to,from,sz(ed[from])-1,0,-cost,0 });
  } // c71528
  void path(int s) {
    fill(all(seen), 0);
    fill(all(dist), INF);
    dist[s] = 0; ll di;
    __gnu_pbds::priority_queue<pair<ll, int>> q;
    vector<decltype(q)::point_iterator> its(N);
    q.push({ 0, s });
    while (!q.empty()) {
      s = q.top().second; q.pop();
      seen[s] = 1; di = dist[s] + pi[s];
      for (edge& e : ed[s]) if (!seen[e.to]) {
```

```
        ll val = di - pi[e.to] + e.cost;
        if (e.cap - e.flow > 0 && val < dist[e.to]) {
          dist[e.to] = val;
          par[e.to] = &e;
          if (its[e.to] == q.end())
            its[e.to] = q.push({ -dist[e.to], e.to });
          else
            q.modify(its[e.to], { -dist[e.to], e.to });
        } // ca07f4
      } // 4cd18f
    } // 062b8f
    rep(i,0,N) pi[i] = min(pi[i] + dist[i], INF);
  } // 7e4cbe
  pair<ll, ll> maxflow(int s, int t) {
    ll totflow = 0, totcost = 0;
    while (path(s), seen[t]) {
      ll fl = INF;
      for (edge* x = par[t]; x; x = par[x->from])
        fl = min(fl, x->cap - x->flow);

      totflow += fl;
      for (edge* x = par[t]; x; x = par[x->from]) {
        x->flow += fl;
        ed[x->to][x->rev].flow -= fl;
      } // 3bfaf3
    } // 8d9a6a
    rep(i,0,N) for(edge& e : ed[i]) totcost += e.cost * e.flow;
    return {totflow, totcost/2};
  } // 24f5a0
  // If some costs can be negative, call this before maxflow:
  void setpi(int s) { // (otherwise, leave this out)
    fill(all(pi), INF); pi[s] = 0;
    int it = N, ch = 1; ll v;
    while (ch-- && it--)
      rep(i,0,N) if (pi[i] != INF)
        for (edge& e : ed[i]) if (e.cap)
          if ((v = pi[i] + e.cost) < pi[e.to])
            pi[e.to] = v, ch = 1;
    assert(it >= 0); // negative cost cycle
  } // 6847d8
}; // b3692f
```

GlobalMinCut.h

Description: Find a global minimum cut in an undirected graph, as represented by an adjacency matrix.

Time: $\mathcal{O}\left(V^3\right)$

8b0e19, 21L, 3m50s

```
pair<int, vi> globalMinCut(vector<vi> mat) {
  pair<int, vi> best = {INT_MAX, {}};
  int n = sz(mat);
  vector<vi> co(n);
  rep(i,0,n) co[i] = {i};
  rep(ph,1,n) {
    vi w = mat[0];
    size_t s = 0, t = 0;
    rep(it,0,n-ph) { // O(V^2) -> O(E log V) with prio. queue
      w[t] = INT_MIN;
      s = t, t = max_element(all(w)) - w.begin();
      rep(i,0,n) w[i] += mat[t][i];
    } // ec93df
    best = min(best, {w[t] - mat[t][t], co[t]});
    co[s].insert(co[s].end(), all(co[t]));
    rep(i,0,n) mat[s][i] += mat[t][i];
    rep(i,0,n) mat[i][s] = mat[s][i];
    mat[0][t] = INT_MIN;
  } // ca0062
  return best;
} // 8b0e19
```

GomoryHu.h

Description: Given a list of edges representing an undirected flow graph, returns edges of the Gomory-Hu tree. The max flow between any pair of vertices is given by minimum edge weight along the Gomory-Hu tree path.

Time: $\mathcal{O}(V)$ Flow Computations

"Dinic.h" e2b333, 13L, 2m20s

```
typedef array<ll, 3> Edge;
vector<Edge> gomoryHu(int N, vector<Edge> ed) {
  vector<Edge> tree;
  vi par(N);
  rep(i,1,N) {
    Dinic D(N); // Dinic also works
    for (Edge t : ed) D.addEdge(t[0], t[1], t[2], t[2]);
    tree.push_back({i, par[i], D.calc(i, par[i])});
    rep(j,i+1,N)
      if (par[j] == par[i] && D.leftOfMinCut(j)) par[j] = i;
  } // 42d832
  return tree;
} // 4edcec
```

5.3 Matching

HopcroftKarp.h

Description: Fast bipartite matching algorithm. Graph g should be a list of neighbors of the left partition, and r should be a vector full of -1's of the same size as the right partition. Returns the size of the matching. $r[i]$ will be the match for vertex i on the right side, or -1 if it's not matched.

Time: $\mathcal{O}\left(E\sqrt{V}\right)$

731cfb, 20L, 4m

```
int hopcroftKarp(vector<vi>& g, vi& r) {
  int n = sz(g), res = 0;
  vi l(n, -1), q(n), d(n);
  auto dfs = [&](auto f, int u) -> bool {
    int t = exchange(d[u], 0) + 1;
    for (int v : g[u])
      if (r[v] == -1 || (d[r[v]] == t && f(f, r[v])))
        return l[u] = v, r[v] = u, 1;
    return 0;
  }; // a95e38
  for (int t = 0, f = 0;; t = f = 0, d.assign(n, 0)) {
    rep(i,0,n) if (l[i] == -1) q[t++] = i, d[i] = 1;
    rep(i,0,t) for (int v : g[q[i]]) {
      if (r[v] == -1) f = 1;
      else if (!d[r[v]]) d[r[v]] = d[q[i]] + 1, q[t++] = r[v];
    } // 64af74
    if (!f) return res;
    rep(i,0,n) if (l[i] == -1) res += dfs(dfs, i);
  } // cdf3b2
} // 731cfb
```

MinimumVertexCover.h

Description: Finds a minimum vertex cover in a bipartite graph. The size is the same as the size of a maximum matching, and the complement is a maximum independent set.

"HopcroftKarp.h" bfb654, 20L, 3m50s

```
vi cover(vector<vi>& g, int n, int m) {
  vi match(m, -1);
  int res = hopcroftKarp(g, match);
  vector<bool> lfound(n, true), seen(m);
  for (int it : match) if (it != -1) lfound[it] = false;
  vi q, cover;
  rep(i,0,n) if (lfound[i]) q.push_back(i);
  while (!q.empty()) {
    int i = q.back(); q.pop_back();
    lfound[i] = 1;
    for (int e : g[i]) if (!seen[e] && match[e] != -1) {
      seen[e] = true;
      q.push_back(match[e]);
    } // 46e035
  } // 069994
  rep(i,0,n) if (!lfound[i]) cover.push_back(i);
  rep(i,0,m) if (seen[i]) cover.push_back(n+i);
  assert(sz(cover) == res);
  return cover;
} // bfb654
```

GeneralMatching.h

Description: Matching for general graphs. 1-indexed vertices.

Memory: $\mathcal{O}(n+m)$

Time: $\mathcal{O}\left(\sqrt{n}m\log_{\max\{2,1+m/n\}}n\right)$

9437b5, 313L, 75m

```
class MaximumMatching {
  public:
   struct Edge {int from, to;};
   static constexpr int Inf = 1 << 30;
  private:
   enum Label {
       kInner = -1,  // should be < 0
       kFree = 0     // should be 0
   }; // c09e7e
   struct Link {int from, to;};
   struct Log {int v, par;};
   struct LinkedList {
       LinkedList() {}
       LinkedList(int N, int M) : N(N), next(M) { clear(); }
       void clear() { head.assign(N, -1); }
       void push(int h, int u) { next[u] = head[h], head[h] =
           u; }
       int N; vector<int> head, next;
   }; // 1a51c7
   template <typename T> struct Queue {
       Queue() {}
       Queue(int N) : qh(0), qt(0), data(N) {}
       T operator[](int i) const { return data[i]; }
       void enqueue(int u) { data[qt++] = u; }
       int dequeue() { return data[qh++]; }
       bool empty() const { return qh == qt; }
       void clear() { qh = qt = 0; }
       int size() const { return qt; }
       int qh, qt; vector<T> data;
   }; // 4bfceb
   struct DisjointSetUnion {
       DisjointSetUnion() {}
       DisjointSetUnion(int N) : par(N) { for (int i = 0; i <
           N; ++i) par[i] = i; }
       int find(int u) { return par[u] == u ? u : (par[u] =
           find(par[u])); }
       void unite(int u, int v) {
           u = find(u), v = find(v);
           if (u != v) par[v] = u;
       } // 461e55
       vector<int> par;
   }; // a21f68
  public:
   MaximumMatching(int N, const vector<Edge> &in)
       : N(N), NH(N >> 1), ofs(N + 2, 0), edges(in.size() * 2)
           {
       for (auto &e : in) ofs[e.from + 1] += 1, ofs[e.to + 1]
           += 1;
       for (int i = 1; i <= N + 1; ++i) ofs[i] += ofs[i - 1];
       for (auto &e : in) {
           edges[ofs[e.from]++] = e;
           edges[ofs[e.to]++] = {e.to, e.from};
       } // 36bba8
       for (int i = N + 1; i > 0; --i) ofs[i] = ofs[i - 1];
       ofs[0] = 0;
```

```
  } // a78156
  int maximum_matching() {
    initialize(); int match = 0;
    while (match * 2 + 1 < N) {
      reset_count();
      bool has_augmenting_path = do_edmonds_search();
      if (!has_augmenting_path) break;
      match += find_maximal();
      clear();
    } // 8c0a2c
    return match;
  } // da7b99
  vector<Edge> get_edges() {
    vector<Edge> ans;
    for (int i = 1; i <= N; ++i) if (mate[i] > i) ans.
        push_back(Edge{i, mate[i]});
    return ans;
  } // f8586b
 private:
  void reset_count() {
    time_current_ = 0;
    time_augment_ = Inf;
    contract_count_ = 0;
    outer_id_ = 1;
    dsu_changelog_size_ = dsu_changelog_last_ = 0;
  } // 6c73d5
  void clear() {
    que.clear();
    for (int u = 1; u <= N; ++u) potential[u] = 1;
    for (int u = 1; u <= N; ++u) dsu.par[u] = u;
    for (int t = time_current_; t <= N / 2; ++t) list.head[
        t] = -1;
    for (int u = 1; u <= N; ++u) blossom.head[u] = -1;
  } // 6e020f
  inline void grow(int x, int y, int z) {
    label[y] = kInner;
    potential[y] = time_current_;  // visited time
    link[z] = {x, y};
    label[z] = label[x];
    potential[z] = time_current_ + 1;
    que.enqueue(z);
  } // 5a596b
  void contract(int x, int y) {
    int bx = dsu.find(x), by = dsu.find(y);
    const int h = -(++contract_count_) + kInner;
    label[mate[bx]] = label[mate[by]] = h;
    int lca = -1;
    while (1) {
      if (mate[by] != 0) swap(bx, by);
      bx = lca = dsu.find(link[bx].from);
      if (label[mate[bx]] == h) break;
      label[mate[bx]] = h;
    } // d03cf2
    for (auto bv : {dsu.par[x], dsu.par[y]}) {
      for (; bv != lca; bv = dsu.par[link[bv].from]) {
        int mv = mate[bv];
        link[mv] = {x, y};
        label[mv] = label[x];
        potential[mv] =
          1 + (time_current_ - potential[mv]) +
            time_current_;
        que.enqueue(mv);
        dsu.par[bv] = dsu.par[mv] = lca;
        dsu_changelog[dsu_changelog_last_++] = {bv, lca
            };
        dsu_changelog[dsu_changelog_last_++] = {mv, lca
            };
      } // 8fef02
    } // 2b0c42
  } // a9547e
  bool find_augmenting_path() {
    while (!que.empty()) {
      int x = que.dequeue(), lx = label[x], px =
          potential[x],
        bx = dsu.find(x);
      for (int eid = ofs[x]; eid < ofs[x + 1]; ++eid) {
        int y = edges[eid].to;
        if (label[y] > 0) {  // outer blossom/vertex
          int time_next = (px + potential[y]) >> 1;
          if (lx != label[y]) {
            if (time_next == time_current_) return
                true;
            time_augment_ = min(time_next,
                time_augment_);
          } else { // a6d4c5
            if (bx == dsu.find(y)) continue;
            if (time_next == time_current_) {
              contract(x, y);
              bx = dsu.find(x);
            } else if (time_next <= NH) // 1efde3
              list.push(time_next, eid);
          } // 1666ad
        } else if (label[y] == kFree) {  // free vertex
            // c17e44
          int time_next = px + 1;
          if (time_next == time_current_)
            grow(x, y, mate[y]);
          else if (time_next <= NH)
            list.push(time_next, eid);
        } // 48780d
      } // 30c821
    } // 1f7dd0
    return false;
  } // 5f1707
  bool adjust_dual_variables() {
    const int time_lim = min(NH + 1, time_augment_);
    for (++time_current_; time_current_ <= time_lim; ++
        time_current_) {
      dsu_changelog_size_ = dsu_changelog_last_;
      if (time_current_ == time_lim) break;
      bool updated = false;
      for (int h = list.head[time_current_]; h >= 0; h =
          list.next[h]) {
        auto &e = edges[h];
        int x = e.from, y = e.to;
        if (label[y] > 0) {
          if (potential[x] + potential[y] != (
              time_current_ << 1))
            continue;
          if (dsu.find(x) == dsu.find(y)) continue;
          if (label[x] != label[y]) {
            time_augment_ = time_current_;
            return false;
          } // e7c864
          contract(x, y);
          updated = true;
        } else if (label[y] == kFree) { // 1a7dca
          grow(x, y, mate[y]);
          updated = true;
        } // ebac0d
      } // 7bf910
      list.head[time_current_] = -1;
      if (updated) return false;
    } // a2963f
    return time_current_ > NH;
  } // 361891
  bool do_edmonds_search() {
    label[0] = kFree;
    for (int u = 1; u <= N; ++u) {
      if (mate[u] == 0) {
        que.enqueue(u);
        label[u] = u;  // component id
      } else // 52e61c
        label[u] = kFree;
    } // 585bc4
    while (1) {
      if (find_augmenting_path()) break;
      bool maximum = adjust_dual_variables();
      if (maximum) return false;
      if (time_current_ == time_augment_) break;
    } // 3b82c1
    for (int u = 1; u <= N; ++u) {
      if (label[u] > 0)
        potential[u] -= time_current_;
      else if (label[u] < 0)
        potential[u] = 1 + (time_current_ - potential[u
            ]);
    } // c77cf7
    return true;
  } // e1fc96
  void rematch(int v, int w) {
    int t = mate[v];
    mate[v] = w;
    if (mate[t] != v) return;
    if (link[v].to == dsu.find(link[v].to)) {
      mate[t] = link[v].from;
      rematch(mate[t], t);
    } else { // 45eff2
      int x = link[v].from, y = link[v].to;
      rematch(x, y);
      rematch(y, x);
    } // 4a404f
  } // c4e2e9
  bool dfs_augment(int x, int bx) {
    int px = potential[x], lx = label[bx];
    for (int eid = ofs[x]; eid < ofs[x + 1]; ++eid) {
      int y = edges[eid].to;
      if (px + potential[y] != 0) continue;
      int by = dsu.find(y), ly = label[by];
      if (ly > 0) {  // outer
        if (lx >= ly) continue;
        int stack_beg = stack_last_;
        for (int bv = by; bv != bx; bv = dsu.find(link[
            bv].from)) {
          int bw = dsu.find(mate[bv]);
          stack[stack_last_++] = bw;
          link[bw] = {x, y};
          dsu.par[bv] = dsu.par[bw] = bx;
        } // 211d3e
        while (stack_last_ > stack_beg) {
          int bv = stack[--stack_last_];
          for (int v = blossom.head[bv]; v >= 0;
               v = blossom.next[v]) {
            if (!dfs_augment(v, bx)) continue;
            stack_last_ = stack_beg;
            return true;
          } // 9ded5e
        } // 476c04
      } else if (ly == kFree) { // e5357f
        label[by] = kInner;
        int z = mate[by];
        if (z == 0) {
          rematch(x, y);
          rematch(y, x);
          return true;
        } // 781371
        int bz = dsu.find(z);
```

```
                link[bz] = {x, y};
                label[bz] = outer_id_++;
                for (int v = blossom.head[bz]; v >= 0; v =
                     blossom.next[v]) {
                    if (dfs_augment(v, bz)) return true;
                } // 5ea9e4
            } // 1c508f
        } // 7f2ab3
        return false;
    } // 44c41a
    int find_maximal() {
        for (int u = 1; u <= N; ++u) dsu.par[u] = u;
        for (int i = 0; i < dsu_changelog_size_; ++i) {
            dsu.par[dsu_changelog[i].v] = dsu_changelog[i].par;
        } // 384510
        for (int u = 1; u <= N; ++u) {
            label[u] = kFree;
            blossom.push(dsu.find(u), u);
        } // 2a02fd
        int ret = 0;
        for (int u = 1; u <= N; ++u)
            if (!mate[u]) {
                int bu = dsu.par[u];
                if (label[bu] != kFree) continue;
                label[bu] = outer_id_++;
                for (int v = blossom.head[bu]; v >= 0; v =
                     blossom.next[v]) {
                    if (!dfs_augment(v, bu)) continue;
                    ret += 1;
                    break;
                } // e4fb6a
            } // 4db9e3
        assert(ret >= 1); return ret;
    } // 500171
    void initialize() {
        que = Queue<int>(N);
        mate.assign(N + 1, 0);
        potential.assign(N + 1, 1);
        label.assign(N + 1, kFree);
        link.assign(N + 1, {0, 0});
        dsu_changelog.resize(N);
        dsu = DisjointSetUnion(N + 1);
        list = LinkedList(NH + 1, edges.size());
        blossom = LinkedList(N + 1, N + 1);
        stack.resize(N);
        stack_last_ = 0;
    } // be011c
   private:
    const int N, NH;
    vector<int> ofs;
    vector<Edge> edges;
    Queue<int> que;
    vector<int> mate, potential;
    vector<int> label;
    vector<Link> link;
    vector<Log> dsu_changelog;
    int dsu_changelog_last_, dsu_changelog_size_;
    DisjointSetUnion dsu;
    LinkedList list, blossom;
    vector<int> stack;
    int stack_last_;
    int time_current_, time_augment_;
    int contract_count_, outer_id_;
}; // b7724d
using Edge = MaximumMatching::Edge;
void example() { // Graph of Love problem
    int n; cin >> n;
    vector<Edge> edges(n, {0, 0});
    for (int i = 1, j; i <= n; i++)cin >> j, edges[i-1] = {i, j
         };
    auto M = MaximumMatching(n, edges);
    cout << M.maximum_matching() << '\n';
} // 0523ec
```

OnlineMatching.h

Description: Modified khun developed for specific question able to run $2*10^6$ queries, in $2*10^6$ x 10^6 graph in 3 seconds codeforces
Time: $\mathcal{O}(confia)$

6ac539, 36L, 5m20s

```
struct OnlineMatching {
    int n = 0, m = 0;
    vector<int> vis, match, dist;
    vector<vector<int>> g;
  vector<int> last;
  int t = 0;
    OnlineMatching(int n_, int m_) : n(n_), m(m_), vis(n, 0),
    match(m, -1), dist(n, n+1), g(n), last(n, -1) {}
    void add(int a, int b) { g[a].pb(b); }
    bool kuhn(int a) {
    vis[a] = t;
    for(int b: g[a]) {
        int c = match[b];
        if (c == -1) {
        match[b] = a;
        return true;
        } // 38b210
        if (last[c] != t || (dist[a] + 1 < dist[c]))
        dist[c] = dist[a] + 1, last[c] = t;
    } // d30675
    for (int b: g[a]) {
      int c = match[b];
      if (dist[a] + 1 == dist[c] && vis[c] != t && kuhn(c)) {
        match[b] = a;
        return true;
      } // 2dac75
    } // e58bd5
        return false;
    } // b533ee
  bool can_match(int a) {
    t++;
    last[a] = t;
    dist[a] = 0;
    return kuhn(a);
  } // 32302b
}; // 6ac539
```

5.4 DFS algorithms

SCC.h

Description: Finds strongly connected components in a directed graph. If vertices u, v belong to the same component, we can reach u from v and vice versa.
Usage: `scc(graph, [&](vi& v) { ... })` visits all components in reverse topological order. `comp[i]` holds the component index of a node (a component only has edges to components with lower index). `ncomps` will contain the number of components.
Time: $\mathcal{O}(E+V)$

76b5c9, 24L, 3m50s

```
vi val, comp, z, cont;
int Time, ncomps;
template<class G, class F> int dfs(int j, G& g, F& f) {
  int low = val[j] = ++Time, x; z.push_back(j);
  for (auto e : g[j]) if (comp[e] < 0)
    low = min(low, val[e] ?: dfs(e,g,f));

  if (low == val[j]) {
    do {
      x = z.back(); z.pop_back();
      comp[x] = ncomps;
      cont.push_back(x);
    } while (x != j); // ae85bd
    f(cont); cont.clear();
    ncomps++;
  } // 64c1b9
  return val[j] = low;
} // 3513bd
template<class G, class F> void scc(G& g, F f) {
  int n = sz(g);
  val.assign(n, 0); comp.assign(n, -1);
  Time = ncomps = 0;
  rep(i,0,n) if (comp[i] < 0) dfs(i, g, f);
} // 56b050
```

BiconnectedComponents.h

Description: Finds all biconnected components in an undirected graph. In a biconnected component there are at least two internally disjoint paths between any two nodes a cycle exists through them. Note that a node can be in several components. Every edge is in a single component. Nodes without edges will not be in any components
Time: $\mathcal{O}(E+V)$

61b438, 50L, 8m

```
vector<pair<int, int>> edges; // edges
vector<vector<pair<int, int>>> g; // [b, edge idx]
vi tin, st, art;
int dfstime = 0;
vector<vi> bcc;
int dfs(int a, int p) {
  int top = tin[a] = ++dfstime;
  bool child = (p != -1);
  for (auto [b, e]: g[a]) {
    if (tin[b]) {
      top = min(top, tin[b]);
      if (tin[b] < tin[a]) st.pb(e);
    } // 0df373
    else {
      int si = sz(st);
      int up = dfs(b, e);
      top = min(top, up);
      if (up > tin[a]) { /*e is a bridge */}
      if (up >= tin[a]) {
        st.pb(e);
        bcc.eb(st.begin() + si, st.end());
        st.resize(si);
        art[a] += child;
        child = true;
      } // 46103a
      else if (up < tin[a]) st.pb(e);
    } // 38ea8a
  } // 53a60e
  return top;
} // 17df2f
void bicomps() {
  int n = sz(g);
  tin.assign(n, 0), art.assign(n, 0);
  rep(i,0,n) if (!tin[i]) dfs(i, -1);
} // 600614
vi comp;
vector<vi> tree;
void build_tree() { // Optional
  int n = sz(g);
  comp.resize(n), tree.resize(n + sz(bcc));
  rep(i, 0, sz(bcc)) {
    for (int eid: bcc[i]) {
      auto [a, b] = edges[eid];
      if (art[a] && (empty(tree[a]) || tree[a].back() != n+i))
          tree[a].pb(n+i), tree[n+i].pb(a);
```

```
      if (art[b] && (empty(tree[b]) || tree[b].back() != n+i))
        tree[b].pb(n+i), tree[n+i].pb(b);
      comp[a] = comp[b] = n + i;
    } // 2e7ad5
  } // 8e4c66
  rep(i, 0, n) if (art[i]) comp[i] = i;
} // b645e9
```

TwoCC.h

Description: Finds all two edge connected components in an undirected graph.
Time: $\mathcal{O}(E+V)$

af5478, 41L, 6m

```
vector<vector<pair<int, int>>> g;
vector<pair<int, int>> edges;
vi tin, st, bridges;
int dfstime = 0;
vector<vi> twocc;
int dfs(int a, int p) {
  int top = tin[a] = ++dfstime;
  int si = st.size();
  st.pb(a);
  for (auto [b, e]: g[a]) if (e != p) {
    if (tin[b]) top = min(top, tin[b]);
    else {
      int up = dfs(b, e);
      top = min(top, up);
      if (up > tin[a]) bridges.pb(e);
    } // 1dd281
  } // ada038
  if (top == tin[a]) {
    twocc.eb(st.begin() + si, st.end());
    st.resize(si);
  } // 0f03f3
  return top;
} // 55bb17
void twocomps() {
  int n = sz(g);
  tin.assign(n, 0);
  rep(i, 0, n) if (!tin[i]) dfs(i, -1);
} // 9a2d22
vi comp;
vector<vi> tree;
void build_tree() { // Optional
  int n = sz(g);
  tree.resize(sz(twocc)); comp.resize(n);
  rep(i, 0, sz(twocc))
    for (int a: twocc[i]) comp[a] = i;

  for (int eid: bridges) {
    auto [a, b] = edges[eid];
    tree[comp[a]].pb(comp[b]), tree[comp[b]].pb(comp[a]);
  } // 1a7e1f
} // c306f0
```

2sat.h

Description: Calculates a valid assignment to boolean variables a, b, c,... to a 2-SAT problem, so that an expression of the type $(a||b)\&\&(!a||c)\&\&(d||!b)\&\&...$ becomes true, or reports that it is unsatisfiable. Negated variables are represented by bit-inversions (~x).
Usage: TwoSat ts(number of boolean variables);

```
ts.either(0, ~3); // Var 0 is true or var 3 is false
ts.setValue(2); // Var 2 is true
ts.atMostOne({0,~1,2}); // <= 1 of vars 0, ~1 and 2 are true
ts.solve(); // Returns true iff it is solvable
ts.values[0..N-1] holds the assigned values to the vars
```

Time: $\mathcal{O}(N+E)$, where N is the number of boolean variables, and E is the number of clauses.

8e1208, 48L, 7m40s

```
struct TwoSat {
  int N; vector<vi> gr; vi values; // 0 = false, 1 = true
  TwoSat(int n = 0) : N(n), gr(2*n) {}
  void either(int f, int j) {
    f = max(2*f, -1-2*f);
    j = max(2*j, -1-2*j);
    gr[f].push_back(j^1);
    gr[j].push_back(f^1);
  } // 516db0
  void setValue(int x) { either(x, x); }
    int addVar() { // (optional)
    gr.emplace_back();
    gr.emplace_back();
    return N++;
  } // 7b5f84
  void atMostOne(const vi& li) { // (optional)
    if (sz(li) <= 1) return;
    int cur = ~li[0];
    rep(i,2,sz(li)) {
      int next = addVar();
      either(cur, ~li[i]);
      either(cur, next);
      either(~li[i], next);
      cur = ~next;
    } // 8d3782
    either(cur, ~li[1]);
  } // 10f2ea
  vi val, comp, z; int time = 0;
  int dfs(int i) {
    int low = val[i] = ++time, x; z.push_back(i);
    for(int e : gr[i]) if (!comp[e])
      low = min(low, val[e] ?: dfs(e));
    if (low == val[i]) do {
      x = z.back(); z.pop_back();
      comp[x] = low;
      if (values[x>>1] == -1)
        values[x>>1] = x&1;
    } while (x != i); // b15351
    return val[i] = low;
  } // ef583a
  bool solve() {
    values.assign(N, -1);
    val.assign(2*N, 0); comp = val;
    rep(i,0,2*N) if (!comp[i]) dfs(i);
    rep(i,0,N) if (comp[2*i] == comp[2*i+1]) return 0;
    return 1;
  } // 2bb76d
}; // 8e1208
```

EulerWalk.h

Description: Eulerian undirected/directed path/cycle algorithm. Input should be a vector of (dest, global edge index), where for undirected graphs, forward/backward edges have the same index. Returns a list of nodes in the Eulerian path/cycle with src at both start and end, or empty list if no cycle/path exists. To get edge indices back, add .second to s and ret.
Time: $\mathcal{O}(V+E)$

780b64, 15L, 3m30s

```
vi eulerWalk(vector<vector<pii>>& gr, int nedges, int src=0) {
  int n = sz(gr);
  vi D(n), its(n), eu(nedges), ret, s = {src};
  D[src]++; // to allow Euler paths, not just cycles
  while (!s.empty()) {
    int x = s.back(), y, e, &it = its[x], end = sz(gr[x]);
    if (it == end){ ret.push_back(x); s.pop_back(); continue; }
    tie(y, e) = gr[x][it++];
    if (!eu[e]) {
      D[x]--, D[y]++;
      eu[e] = 1; s.push_back(y);
    }} // 22a87a // 94de26
  for (int x : D) if (x < 0 || sz(ret) != nedges+1) return {};
  return {ret.rbegin(), ret.rend()};
} // 780b64
```

DominatorTree.h

Description: Dominator Tree, creates the graph tree, where all ancestors of a u in the tree are necessary in the path from the root to u
Memory: $\mathcal{O}(n)$
Time: $\mathcal{O}((n+m)log(n))$ build

d7c79f, 54L, 9m10s

```
struct DominatorTree {
  int n, dfstime;
  vector<vector<int>> g, gt, tree, bucket, down;
  vector<int> S,  dsu, label, sdom, idom, id;

  DominatorTree(vector<vector<int>> & _g, int root)
    : n(sz(_g)), dfstime(0), g(_g), gt(n), tree(n), bucket(n),
        down(n),
    S(n), dsu(n), label(n), sdom(n), idom(n), id(n) {
    prep(root); reverse(S.begin(), S.begin() + dfstime);
    for(int u : S) {
      for(int v : gt[u]) {
        int w = fnd(v);
        if(id[ sdom[w] ] < id[ sdom[u] ])
          sdom[u] = sdom[w];
      } // e059b2
      gt[u].clear();
      if(u != root) bucket[ sdom[u] ].push_back(u);
      for(int v : bucket[u]) {
        int w = fnd(v);
        if(sdom[w] == sdom[v]) idom[v] = sdom[v];
        else idom[v] = w;
      } // 72077b
      bucket[u].clear();
      for(int v : down[u]) dsu[v] = u;
      down[u].clear();
    } // 3197c4
    reverse(S.begin(), S.begin() + dfstime);
    for(int u : S) if(u != root) {
      if(idom[u] != sdom[u]) idom[u] = idom[ idom[u] ];
      tree[ idom[u] ].push_back(u);
    } // 96e582
    idom[root] = root;
  } // b239ba
  void prep(int u){
    S[dfstime] = u;
    id[u] = ++dfstime;
    label[u] = sdom[u] = dsu[u] = u;

    for(int v : g[u]){
      if(!id[v])
        prep(v), down[u].push_back(v);
      gt[v].push_back(u);
    } // 4d7944
  } // 4351b9
  int fnd(int u, int flag = 0){
    if(u == dsu[u]) return u;
    int v = fnd(dsu[u], 1), b = label[ dsu[u] ];
    if(id[ sdom[b] ] < id[ sdom[ label[u] ] ])
      label[u] = b;
    dsu[u] = v;
    return flag ? v : label[u];
  } // d64927

}; // d7c79f
```

5.5 Cliques

MaximalCliques.h

Description: Runs a callback for all maximal cliques in a graph (given as a symmetric bitset matrix; self-edges not allowed). Callback is given a bitset representing the maximal clique.

Time: $\mathcal{O}\left(3^{n/3}\right)$, much faster for sparse graphs

b0d5b1, 12L, 2m10s

```
typedef bitset<128> B;
template<class F>
void cliques(vector<B>& eds, F f, B P = ~B(), B X={}, B R={}) {
  if (!P.any()) { if (!X.any()) f(R); return; }
  auto q = (P | X)._Find_first();
  auto cands = P & ~eds[q];
  rep(i,0,sz(eds)) if (cands[i]) {
    R[i] = 1;
    cliques(eds, f, P & eds[i], X & eds[i], R);
    R[i] = P[i] = 0; X[i] = 1;
  } // 181f8f
} // c9dc5f
```

MaximumClique.h

Description: Quickly finds a maximum clique of a graph (given as symmetric bitset matrix; self-edges not allowed). Can be used to find a maximum independent set by finding a clique of the complement graph.

Time: Runs in about 1s for n=155 and worst case random graphs (p=.90). Runs faster for sparse graphs.

f7c0bc, 49L, 9m30s

```
typedef vector<bitset<200>> vb;
struct Maxclique {
  double limit=0.025, pk=0;
  struct Vertex { int i, d=0; };
  typedef vector<Vertex> vv;
  vb e;
  vv V;
  vector<vi> C;
  vi qmax, q, S, old;
  void init(vv& r) {
    for (auto& v : r) v.d = 0;
    for (auto& v : r) for (auto j : r) v.d += e[v.i][j.i];
    sort(all(r), [](auto a, auto b) { return a.d > b.d; });
    int mxD = r[0].d;
    rep(i,0,sz(r)) r[i].d = min(i, mxD) + 1;
  } // 7c428e
  void expand(vv& R, int lev = 1) {
    S[lev] += S[lev - 1] - old[lev];
    old[lev] = S[lev - 1];
    while (sz(R)) {
      if (sz(q) + R.back().d <= sz(qmax)) return;
      q.push_back(R.back().i);
      vv T;
      for(auto v:R) if (e[R.back().i][v.i]) T.push_back({v.i});
      if (sz(T)) {
        if (S[lev]++ / ++pk < limit) init(T);
        int j = 0, mxk = 1, mnk = max(sz(qmax) - sz(q) + 1, 1);
        C[1].clear(), C[2].clear();
        for (auto v : T) {
          int k = 1;
          auto f = [&](int i) { return e[v.i][i]; };
          while (any_of(all(C[k]), f)) k++;
          if (k > mxk) mxk = k, C[mxk + 1].clear();
          if (k < mnk) T[j++].i = v.i;
          C[k].push_back(v.i);
        } // 3221ac
        if (j > 0) T[j - 1].d = 0;
        rep(k,mnk,mxk + 1) for (int i : C[k])
          T[j].i = i, T[j++].d = k;
        expand(T, lev + 1);
      } else if (sz(q) > sz(qmax)) qmax = q; // 2a0537
      q.pop_back(), R.pop_back();
    } // 87639b
  } // f0a49d
  vi maxClique() { init(V), expand(V); return qmax; }
  Maxclique(vb conn) : e(conn), C(sz(e)+1), S(sz(C)), old(S) {
    rep(i,0,sz(e)) V.push_back({i});
  } // 36accb
}; // b63641
```

5.6 Trees

CompressTree.h

Description: Given a rooted tree and a subset S of nodes, compute the minimal subtree that contains all the nodes by adding all (at most $|S|-1$) pairwise LCA's and compressing edges. Returns a list of (par, orig_index) representing a tree rooted at 0. The root points to itself.

Time: $\mathcal{O}(|S|\log|S|)$

7347c3, 26L, 4m10s

```
struct LCA {
  vi time;
  int lca(int a, int b) { return -1; }
}; // 32d408

typedef vector<pair<int, int>> vpi;
vpi compressTree(LCA& lca, const vi& subset) {
  static vi rev; rev.resize(sz(lca.time));
  vi li = subset, &T = lca.time;
  auto cmp = [&](int a, int b) { return T[a] < T[b]; };
  sort(all(li), cmp);
  int m = sz(li)-1;
  rep(i,0,m) {
    int a = li[i], b = li[i+1];
    li.push_back(lca.lca(a, b));
  } // 677c62
  sort(all(li), cmp);
  li.erase(unique(all(li)), li.end());
  rep(i,0,sz(li)) rev[li[i]] = i;
  vpi ret = {pii(0, li[0])};
  rep(i,0,sz(li)-1) {
    int a = li[i], b = li[i+1];
    ret.emplace_back(rev[lca.lca(a, b)], b);
  } // 5efe90
  return ret;
} // 83c9a2
```

HLD.h

Description: Does the same thing as a LazySegment tree, but on a tree and with an extra log factor to the complexity. VAL_EDGES determines if the values are stored in edges. If so, store the value of each edge on its child vertex.

Time: $\mathcal{O}\left((\log N)^2\right)$ for range operations, $\mathcal{O}(log(n))$ for point operations

"../data-structures/LazySegmentTree.h" b7947a, 53L, 9m40s

```
template<class S, bool VAL_EDGES>
struct HLD {
  using T = typename S::T; using L = typename S::L;
  int n,tim=0;
  vector<vi> g;
  vi p,siz,hd,ti;
  SegBeats<S> seg, segi;
  HLD(vector<vi> ag) : n(sz(ag)), g(ag), p(n), siz(n,1), hd(n),
        ti(n),
  seg(n), segi(n) { dfs(0); dfs2(0);}
  #warning fix n to be a power of two
  void dfs(int v) {
    for (int& u : g[v]) {
      g[u].erase(find(all(g[u]), v));
      p[u]=v;
      dfs(u);
      siz[v] += siz[u];
      if (siz[u] > siz[g[v][0]]) swap(u, g[v][0]);
    } // 7b54be
  } // a86bcf
  void dfs2(int v) {
    ti[v]=tim++;
    for (int u : g[v]) {
      hd[u] = (u == g[v][0] ? hd[v] : u);
      dfs2(u);
    } // 79707a
  } // 816069
  template <class B, class C> // ops are [l, r)
  void process(int u, int v, B op1, C op2) {
    for (;;) {
      if (hd[u] == hd[v]) break;
      if (ti[u] < ti[v]) op2(ti[hd[v]], ti[v]+1), v=p[hd[v]];
      else op1(ti[hd[u]], ti[u]+1), u=p[hd[u]];
    } // 1cb33f
    if (ti[u] < ti[v]) op2(ti[u]+VAL_EDGES, ti[v]+1);
    else op1(ti[v]+VAL_EDGES, ti[u]+1);
  } // 6a6a62
  T query(int u, int v) { // inclusive on both ends
    T vu = S::id, vv = S::id;
    process(u,v,
      [&](int l, int r){ vu = S::op(vu, segi.query(n-r,n-l)); }
        ,
      [&](int l, int r){ vv = S::op(seg.query(l,r), vv); } );
    return S::op(vu,vv);
  } // 04b81f
  void update(L val, int u, int v) {
    process(u,v,
      [&](int l, int r){ segi.update(val,n-r,n-l); },
      [&](int l, int r){ seg.update(val,l,r);  });
  } // 8e0c4e
  void update(L val, int u) {
    seg.update(val,ti[u], ti[u]+1);
    segi.update(val,n-ti[u]-1, n-ti[u]);
  } // 6fea94
}; // 14746f
```

LinkCutTree.h

Description: Represents a forest of unrooted trees. You can add and remove edges (as long as the result is still a forest), and check whether two nodes are in the same tree.

Time: All operations take amortized $\mathcal{O}(\log N)$.

0fb462, 88L, 13m30s

```
struct Node { // Splay tree. Root's pp contains tree's parent.
  Node *p = 0, *pp = 0, *c[2];
  bool flip = 0;
  Node() { c[0] = c[1] = 0; fix(); }
  void fix() {
    if (c[0]) c[0]->p = this;
    if (c[1]) c[1]->p = this;
    // (+ update sum of subtree elements etc. if wanted)
  } // 454758
  void pushFlip() {
    if (!flip) return;
    flip = 0; swap(c[0], c[1]);
    if (c[0]) c[0]->flip ^= 1;
    if (c[1]) c[1]->flip ^= 1;
  } // 0cc949
  int up() { return p ? p->c[1] == this : -1; }
  void rot(int i, int b) {
    int h = i ^ b;
    Node *x = c[i], *y = b == 2 ? x : x->c[h], *z = b ? y : x;
    if ((y->p = p)) p->c[up()] = y;
    c[i] = z->c[i ^ 1];
    if (b < 2) {
      x->c[h] = y->c[h ^ 1];
      y->c[h ^ 1] = x;
    } // 1a82cf
```

```
    z->c[i ^ 1] = this;
    fix(); x->fix(); y->fix();
    if (p) p->fix();
    swap(pp, y->pp);
  } // 1cf643
  void splay() {
    for (pushFlip(); p; ) {
      if (p->p) p->p->pushFlip();
      p->pushFlip(); pushFlip();
      int c1 = up(), c2 = p->up();
      if (c2 == -1) p->rot(c1, 2);
      else p->p->rot(c2, c1 != c2);
    } // e639f4
  } // bfb1f7
  Node* first() {
    pushFlip();
    return c[0] ? c[0]->first() : (splay(), this);
  } // 67f9a1
}; // 225109
struct LinkCut {
  vector<Node> node;
  LinkCut(int N) : node(N) {}
  void link(int u, int v) { // add an edge (u, v)
    assert(!connected(u, v));
    makeRoot(&node[u]);
    node[u].pp = &node[v];
  } // 60799e
  void cut(int u, int v) { // remove an edge (u, v)
    Node *x = &node[u], *top = &node[v];
    makeRoot(top); x->splay();
    assert(top == (x->pp ?: x->c[0]));
    if (x->pp) x->pp = 0;
    else {
      x->c[0] = top->p = 0;
      x->fix();
    } // 8acbe8
  } // a58ec7
  bool connected(int u, int v) { // are u, v in the same tree?
    Node* nu = access(&node[u])->first();
    return nu == access(&node[v])->first();
  } // b80a22
  void makeRoot(Node* u) {
    access(u);
    u->splay();
    if(u->c[0]) {
      u->c[0]->p = 0;
      u->c[0]->flip ^= 1;
      u->c[0]->pp = u;
      u->c[0] = 0;
      u->fix();
    } // 586a65
  } // 74c908
  Node* access(Node* u) {
    u->splay();
    while (Node* pp = u->pp) {
      pp->splay(); u->pp = 0;
      if (pp->c[1]) {
        pp->c[1]->p = 0; pp->c[1]->pp = pp; } // 1ccdfc
      pp->c[1] = u; pp->fix(); u = pp;
    } // b10f33
    return u;
  } // 4ac291
}; // ceab83
```

DirectedMST.h

Description: Finds a minimum spanning tree/arborescence of a directed graph, given a root node. If no MST exists, returns -1.
Time: $\mathcal{O}(E \log V)$

"../data-structures/UnionFindRollback.h" 39e620, 58L, 10m20s

```
struct Edge { int a, b; ll w; };
struct Node {
  Edge key;
  Node *l, *r;
  ll delta;
  void prop() {
    key.w += delta;
    if (l) l->delta += delta;
    if (r) r->delta += delta;
    delta = 0;
  } // 0d348f
  Edge top() { prop(); return key; }
}; // ab4902
Node *merge(Node *a, Node *b) {
  if (!a || !b) return a ?: b;
  a->prop(), b->prop();
  if (a->key.w > b->key.w) swap(a, b);
  swap(a->l, (a->r = merge(b, a->r)));
  return a;
} // c5109e
void pop(Node*& a) { a->prop(); a = merge(a->l, a->r); }
pair<ll, vi> dmst(int n, int r, vector<Edge>& g) {
  RollbackUF uf(n);
  vector<Node*> heap(n);
  for (Edge e : g) heap[e.b] = merge(heap[e.b], new Node{e});
  ll res = 0;
  vi seen(n, -1), path(n), par(n);
  seen[r] = r;
  vector<Edge> Q(n), in(n, {-1,-1}), comp;
  deque<tuple<int, int, vector<Edge>>> cycs;
  rep(s,0,n) {
    int u = s, qi = 0, w;
    while (seen[u] < 0) {
      if (!heap[u]) return {-1,{}};
      Edge e = heap[u]->top();
      heap[u]->delta -= e.w, pop(heap[u]);
      Q[qi] = e, path[qi++] = u, seen[u] = s;
      res += e.w, u = uf.find(e.a);
      if (seen[u] == s) {
        Node* cyc = 0;
        int end = qi, time = uf.time();
        do cyc = merge(cyc, heap[w = path[--qi]]);
        while (uf.join(u, w));
        u = uf.find(u), heap[u] = cyc, seen[u] = -1;
        cycs.push_front({u, time, {&Q[qi], &Q[end]}});
      } // 00a339
    } // c8f0da
    rep(i,0,qi) in[uf.find(Q[i].b)] = Q[i];
  } // fa3c2c
  for (auto& [u,t,comp] : cycs) { // restore sol (optional)
    uf.rollback(t);
    Edge inEdge = in[u];
    for (auto& e : comp) in[uf.find(e.b)] = e;
    in[uf.find(inEdge.b)] = inEdge;
  } // 4f9b56
  rep(i,0,n) par[i] = in[i].a;
  return {res, par};
} // efa3a4
```

SmallToLarge.h

Description: good small to large
Time: $\mathcal{O}(N \log N)$

19475e, 26L, 5m30s

```
void small_to_large(vector<vector<int>> & g){
  int n = sz(g);
  int cans = 0;
  auto add = [&](int v){ cans += v; };
  auto del = [&](int v){ cans -= v; };
  vector ans(n, 0), big(n, -1), sub(n, 1), pos(n, 0), pre(0, 0)
    ;
  auto dfspre = [&](auto&& rec, int v, int p)->void {
    pos[v] = sz(pre); pre.push_back(v);
    for(int f : g[v])if (f != p){
      rec(rec, f, v);
      sub[v] += sub[f];
      if (big[v] == -1 or sub[f] > sub[big[v]])big[v] = f;
    } // 2f5782
  }; // 125df2
  dfspre(dfspre, 0, 0);
  auto dfs = [&](auto&& rec, int v, int p, int keep)->void {
    int b = big[v];
    for(int f : g[v])if (f != p and f != b)rec(rec, f, v, 0);
    if (b != -1)rec(rec, b, v, 1);
    add(v);
    for(int f : g[v])if (f != p and f != b)rep(i,pos[f],pos[f]+
        sub[f])add(pre[i]);
    ans[v] = cans;
    if (not keep)rep(i,pos[v],pos[v]+sub[v])del(pre[i]);
  }; // 022003
  dfs(dfs, 0, 0, 1);
} // 19475e
```

Data structures (6)

6.1 General

HashMap.h

Description: Hash map with mostly the same API as unordered_map, but ~3x faster. Uses 1.5x memory. Initial capacity must be a power of 2 (if provided).

d77092, 7L, 2m

```
#include <bits/extc++.h>
// To use most bits rather than just the lowest ones:
struct chash { // large odd number for C
  const uint64_t C = ll(4e18 * acos(0)) | 71;
  ll operator()(ll x) const { return __builtin_bswap64(x*C); }
}; // cdd37e
__gnu_pbds::gp_hash_table<ll,int,chash> h({},{},{},{},{1<<16});
```

OrderedSet.h

Description: ordered/indexed set and multiset Bad constant factor, works only in Linux
Memory: $\mathcal{O}(N)$
Time: $\mathcal{O}(\log N)$ operations

<bits/extc++.h> //include it before any defines b8c30a, 15L, 4m20s

```
using namespace __gnu_pbds;
template<class T, class B = null_type> using ordered_set = tree
    <T, B, less<T>, rb_tree_tag,
    tree_order_statistics_node_update>;

template<class T>
struct ordered_multiset{
  ordered_set<pair<T, int>> o; int c;
  ordered_multiset():c(0){}
  unsigned order_of_key(T x){return o.order_of_key({x, -1});}
  const T* find_by_order(int p){return &(*o.find_by_order(p)).
      first;}
  void insert(T x){o.insert({x, c++});}
  void erase(T x){o.erase(o.lower_bound({x, 0}));}
  unsigned size(){return o.size();}
```

```
  const T* lower_bound(T x){return &(*o.lower_bound({x, 0})).
      first;}
  const T* upper_bound(T x){return &(*o.upper_bound({x, c})).
      first;}
}; // d4e54f
```

UnionFindRollback.h

Description: Disjoint-set data structure with undo. If undo is not needed, skip st, time() and rollback().
Usage: `int t = uf.time(); ...; uf.rollback(t);`
Time: $\mathcal{O}(\log(N))$

de4ad0, 21L, 3m30s

```
struct RollbackUF {
  vi e; vector<pii> st;
  RollbackUF(int n) : e(n, -1) {}
  int size(int x) { return -e[find(x)]; }
  int find(int x) { return e[x] < 0 ? x : find(e[x]); }
  int time() { return sz(st); }
  void rollback(int t) {
    for (int i = time(); i --> t;)
      e[st[i].first] = st[i].second;
    st.resize(t);
  } // 30bb61
  bool join(int a, int b) {
    a = find(a), b = find(b);
    if (a == b) return false;
    if (e[a] > e[b]) swap(a, b);
    st.push_back({a, e[a]});
    st.push_back({b, e[b]});
    e[a] += e[b]; e[b] = a;
    return true;
  } // 6c709f
}; // de4ad0
```

StaticCHT.h

Description: static CHT - add must be ordered by slope (a), queries by x.
Time: amortized $\mathcal{O}(1)$.

da7e40, 28L, 4m10s

```
struct CHT {
  int it; vector<ll> a, b;
  CHT():it(0){}
  ll eval(int i, ll x){return a[i]*x + b[i];}
  bool useless(){
    int sz = a.size();
    int r = sz-1, m = sz-2, l = sz-3;
#warning careful with overflow!
    return (b[l] - b[r])*(a[m] - a[l]) <
      (b[l] - b[m])*(a[r] - a[l]);
  } // c37135
  void add(ll A, ll B){
    a.push_back(A); b.push_back(B);
    while (!a.empty()){
      if ((a.size() < 3) || !useless()) break;
      a.erase(a.end() - 2);
      b.erase(b.end() - 2);
    } // b21fc8
    it = min(it, int(a.size()) - 1);
  } // 6df532
  ll get(ll x){
    while (it+1 < a.size()){
      if (eval(it+1, x) > eval(it, x)) it++;
      else break;
    } // fe9dba
    return eval(it, x);
  } // b44949
}; // da7e40
```

LineContainer.h

Description: Container where you can add lines of the form kx+m, and query maximum values at points x. Useful for dynamic programming ("convex hull trick").
Time: $\mathcal{O}(\log N)$

8ec1c7, 29L, 6m

```
struct Line {
  mutable ll k, m, p;
  bool operator<(const Line& o) const { return k < o.k; }
  bool operator<(ll x) const { return p < x; }
}; // 7e3ecf
struct LineContainer : multiset<Line, less<>> {
  // (for doubles, use inf = 1/.0, div(a,b) = a/b)
  static const ll inf = LLONG_MAX;
  ll div(ll a, ll b) { // floored division
    return a / b - ((a ^ b) < 0 && a % b); } // 10f081
  bool isect(iterator x, iterator y) {
    if (y == end()) return x->p = inf, 0;
    if (x->k == y->k) x->p = x->m > y->m ? inf : -inf;
    else x->p = div(y->m - x->m, x->k - y->k);
    return x->p >= y->p;
  } // 2fac86
  void add(ll k, ll m) {
    auto z = insert({k, m, 0}), y = z++, x = y;
    while (isect(y, z)) z = erase(z);
    if (x != begin() && isect(--x, y)) isect(x, y = erase(y));
    while ((y = x) != begin() && (--x)->p >= y->p)
      isect(x, erase(y));
  } // 08625f
  ll query(ll x) {
    assert(!empty());
    auto l = *lower_bound(x);
    return l.k * x + l.m;
  } // d21e2f
}; // 5771f0
```

6.2 Range Queries

Spec.h

Description: Spec for Seg Beats. It's similar to other data structures.
Time: varies

30faf7, 32L, 5m10s

```
struct node{int max1,max2,maxc; ll sum;};
struct BeatsSpec{ //chmin sum
  using T = node; using L = int;
  static constexpr T id = node{-oo,-oo,0,0};
  static T op(T a, T b){
    node n;
    if (a.max1 > b.max1){
      n.max1 = a.max1;
      n.max2 = max(a.max2, b.max1);
      n.maxc = a.maxc;
    } // 60044a
    else if (a.max1 == b.max1){
      n.max1 = a.max1;
      n.max2 = max(a.max2, b.max2);
      n.maxc = a.maxc+b.maxc;
    } // 89a062
    else{
      n.max1 = b.max1;
      n.max2 = max(b.max2, a.max1);
      n.maxc = b.maxc;
    } // dbf86a
    n.sum=a.sum+b.sum;
    return n;
  } // 7211b8
  static T ch(T a, L b, int l, int r){
    if (a.max2 <= b)a.sum -= (ll)(a.max1-b)*a.maxc, a.max1 = b;
    return a;
  } // 6f5ec4
  static L cmp(L a, L b){return min(a,b);}
  static bool brk(L a, T b){return b.max1 <= a;}
  static bool tag(L a, T b){return b.max2 < a;}
}; // 0a5678
```

Treap.h

Description: Lazy treap. Its Spec is the same as LazySpec, except that update has the signature update(K f, int n, S a), where n represents the range size.
Time: $\mathcal{O}(\log N)$

ccfd38, 47L, 8m40s

```
#define TT template<typename S>
#define NN node<S>*
mt19937 rng(chrono::steady_clock::now().time_since_epoch().
    count());
TT struct node {
  node *l=0, *r=0;
  int y, c=1;
  typename S::S val, acc;
  typename S::K lz;
  bool hlz=0;
  node(typename S::S aval) :  y(uniform_int_distribution<int
      >(0,(int)1e9)(rng)), val(aval), acc(aval) {}
  void rc();
}; // 2469fa
TT int cnt(S n) {return n ? n-> c : 0;}
TT void prop(NN t) {
  if (!t or !t->hlz) return;
  t->hlz=0;
  t->val = S::update(t->lz, 1, t->val);
  t->acc = S::update(t->lz, t->c, t->acc);
  if (t->l) t->l->lz = t->l->hlz ? S::compose(t->l->lz, t->lz)
      : t->lz, t->l->hlz=1;
  if (t->r) t->r->lz = t->r->hlz ? S::compose(t->r->lz, t->lz)
      : t->lz, t->r->hlz=1;
} // 2a8212
TT void node<S>::rc() {
  c = cnt(l) + cnt(r) + 1;
  prop(l); prop(r);
  acc = S::op(S::op(l ? l->acc : S::id(), val), r ? r->acc : S
      ::id());
} // 25313c
TT pair<NN,NN> split(NN t, int k) {
  if (k == 0) return {0, t};
  if (cnt(t->l) >= k) {
    prop(t->l);
    auto [l, r] = split(t->l, k);
    t->l = r;
    t->rc();
    return {l, t};
  } // b8bc7b
  prop(t->r);
  auto [l,r] = split(t->r, k - cnt(t->l) - 1);
  t->r = l;
  t->rc();
  return {t, r};
} // 480f11
TT auto merge(NN l, NN r) {
  if (!l) return r;
  if (!r) return l;
  if (l->y < r->y) return prop(l->r), l->r = merge(l->r, r), l
      ->rc(), l;
  return prop(r->l), r->l = merge(l, r->l), r->rc(), r;
} // 902057
```

FenwickTree.h

Description: Computes partial sums a[0] + a[1] + ... + a[pos - 1], and updates single elements a[i], taking the difference between the old and new value.

Time: Both operations are $\mathcal{O}(\log N)$.

e62fac, 22L, 4m

```
struct FT {
  vector<ll> s;
  FT(int n) : s(n) {}
  void update(int pos, ll dif) { // a[pos] += dif
    for (; pos < sz(s); pos |= pos + 1) s[pos] += dif;
  } // a388f1
  ll query(int pos) { // sum of values in [0, pos)
    ll res = 0;
    for (; pos > 0; pos &= pos - 1) res += s[pos-1];
    return res;
  } // 6defa0
  int lower_bound(ll sum) {// min pos st sum of [0, pos] >= sum
    // Returns n if no sum is >= sum, or -1 if empty sum is.
    if (sum <= 0) return -1;
    int pos = 0;
    for (int pw = 1 << 25; pw; pw >>= 1) {
      if (pos + pw <= sz(s) && s[pos + pw-1] < sum)
        pos += pw, sum -= s[pos-1];
    } // 63f005
    return pos;
  } // ea70d8
}; // e62fac
```

FenwickTree2d.h

Description: Computes sums a[i,j] for all i<I, j<J, and increases single elements a[i,j]. Requires that the elements to be updated are known in advance (call fakeUpdate() before init()).

Time: $\mathcal{O}\left(\log^2 N\right)$. (Use persistent segment trees for $\mathcal{O}(\log N)$.)

"FenwickTree.h" 157f07, 22L, 3m40s

```
struct FT2 {
  vector<vi> ys; vector<FT> ft;
  FT2(int limx) : ys(limx) {}
  void fakeUpdate(int x, int y) {
    for (; x < sz(ys); x |= x + 1) ys[x].push_back(y);
  } // 01fc7b
  void init() {
    for (vi& v : ys) sort(all(v)), ft.emplace_back(sz(v));
  } // d5ca1f
  int ind(int x, int y) {
    return (int)(lower_bound(all(ys[x]), y) - ys[x].begin()); }
        // aee02d
  void update(int x, int y, ll dif) {
    for (; x < sz(ys); x |= x + 1)
      ft[x].update(ind(x, y), dif);
  } // bb1454
  ll query(int x, int y) {
    ll sum = 0;
    for (; x; x &= x - 1)
      sum += ft[x-1].query(ind(x-1, y));
    return sum;
  } // 8334c3
}; // 157f07
```

MoQueries.h

Description: Answer interval or tree path queries. Includes interval version without deletion. If values are on tree edges, change `step` to add/remove the edge (a, c) and remove the initial `add` call (but keep `in`).

Time: $\mathcal{O}\left(N\sqrt{Q}\right)$

d5377e, 76L, 15m30s

```
void add(int ind, int end) { ... } // add a[ind] (end = 0 or 1)
void del(int ind, int end) { ... } // remove a[ind]
int calc() { ... } // compute current answer

vi mo(vector<pii> Q) {
  int L = 0, R = 0, blk = 350; // ~N/sqrt(Q)
  vi s(sz(Q)), res = s;
#define K(x) pii(x.first/blk, x.second ^ -(x.first/blk & 1))
  iota(all(s), 0);
  sort(all(s), [&](int s, int t){ return K(Q[s]) < K(Q[t]); });
  for (int qi : s) {
    pii q = Q[qi];
    while (L > q.first) add(--L, 0);
    while (R < q.second) add(R++, 1);
    while (L < q.first) del(L++, 0);
    while (R > q.second) del(--R, 1);
    res[qi] = calc();
  } // 0f7fae
  return res;
} // e3731f

vi moTree(vector<array<int, 2>> Q, vector<vi>& ed, int root=0){
  int N = sz(ed), pos[2] = {}, blk = 350; // ~N/sqrt(Q)
  vi s(sz(Q)), res = s, I(N), L(N), R(N), in(N), par(N);
  add(0, 0), in[0] = 1;
  auto dfs = [&](int x, int p, int dep, auto& f) -> void {
    par[x] = p;
    L[x] = N;
    if (dep) I[x] = N++;
    for (int y : ed[x]) if (y != p) f(y, x, !dep, f);
    if (!dep) I[x] = N++;
    R[x] = N;
  }; // 329c88
  dfs(root, -1, 0, dfs);
#define K(x) pii(I[x[0]] / blk, I[x[1]] ^ -(I[x[0]] / blk & 1))
  iota(all(s), 0);
  sort(all(s), [&](int s, int t){ return K(Q[s]) < K(Q[t]); });
  for (int qi : s) rep(end,0,2) {
    int &a = pos[end], b = Q[qi][end], i = 0;
#define step(c) { if (in[c]) { del(a, end); in[a] = 0; } \
                  else { add(c, end); in[c] = 1; } a = c; } //
                      3839ba
    while (!(L[b] <= L[a] && R[a] <= R[b]))
      I[i++] = b, b = par[b];
    while (a != b) step(par[a]);
    while (i--) step(I[i]);
    if (end) res[qi] = calc();
  } // c880be
  return res;
} // ce9c1e

vector mo_no_deletion(vector<pii>& qs, int n){
  int q = sz(qs), sq = (int)sqrt(q)+1, blk = (n+sq+1)/sq;
  vector<vi> o((n+blk-1)/blk);
  rep(i,0,q)o[qs[i].first/blk].pb(i);
  for(auto& vq : o)sort(all(vq), [&](int i, int j){return qs[i
      ].second < qs[j].second;});
  vector<int> ans(q);
  rep(i,0,sz(o)){
    auto& vq = o[i];
    int l = blk*i + blk, r = l-1;
    // prepare to answer queries
    for(int qi : vq){
      auto [ql, qr] = qs[qi];
      if (qr <= l){ //if it does not extrapolate
        rep(j,ql,qr+1)add(j, 1); //solving manually
        continue;
      } // b94913
      while(r < qr)add(++r, 1);
      int ml = l; //we will move l manually
      // prepare checkpoint
      while(ml > ql)add(--ml, 0);
      ans[qi] = calc();
      // revert checkpoint: discard changes made by moving l
    } // 6569ca
  } // 09ff2d
  return ans;
} // fe1e74
```

MergeSortTree.h

Description: Merge Sort Tree

Memory: $\mathcal{O}(N \log N)$

Time: $\mathcal{O}\left(\log^2 N\right)$

7c1cf9, 38L, 6m50s

```
template<class T> struct MGST{
  int n, h; vector<vector<T>> t;
  int lg(int x){return __builtin_clz(1)-__builtin_clz(x);}
  MGST(vector<T> v): n(sz(v)), h(lg(n)){
    if (n != (1<<h))n = 1<<(++h);
    t.assign(h, vector<T>(n));
    rep(i,0,sz(v))t[0][i] = v[i];
    rep(i,sz(v),n)t[0][i] = oo; //non-existent
    rep(k,0,h)for(int i = 0, s = 1<<k; i < n; i += 2*s){
      int p1=0, p2=0;
      rep(p,i,i+2*s){
        if (p1==s)t[k+1][p] = t[k][i+s+p2], p2++;
        else if (p2==s)t[k+1][p] = t[k][i+p1], p1++;
        else if (t[k][i+p1] < t[k][i+s+p2])t[k+1][p] = t[k][i+
            p1], p1++;
        else t[k+1][p] = t[k][i+s+p2], p2++;
      } // 690730
    } // eb4c11
  } // b7e287
  T query_helper(T x, int k, int l){
    auto it = upper_bound(t[k]+l, t[k]+l+(1<<k), x);
    if (it == t[k]+l)return 0;
    else return *prev(it);
  } // ef2397
  T lb(T x, int l, int r){ //biggest <= x in [l, r]
    T ans = 0; r++;
    for(int k = 0; l < r; k++){
      if ((l>>k)&1){
        ans = max(ans, query_helper(x, k, l));
        l += 1<<k;
      } // 1bf09f
      if ((r>>k)&1){
        r -= 1<<k;
        ans = max(ans, query_helper(x, k, r));
      } // aa3bad
    } // b63c49
    return ans;
  } // 8e4027
}; // 7c1cf9
```

MPsum.h

Description: Multidimensional Psum Requires Abelian Group (op, inv, id)

Memory: $\mathcal{O}\left(N^D\right)$

Time: $\mathcal{O}\left(2^D\right)$

3c2845, 20L, 4m30s

```
#define MAs template<class... As> //multiple arguments
template<int D, class S> struct Psum{
  using T = typename S::T;
  int n; vector<Psum<D-1, S>> v;
  MAs Psum(int s, As... ds):n(s+1),v(n,Psum<D-1, S>(ds...)){}
  MAs void set(T x, int p, As... ps){v[p+1].set(x, ps...);}
  void push(Psum& p){rep(i, 1, n)v[i].push(p.v[i]);}
  void init(){rep(i, 1, n)v[i].init(),v[i].push(v[i-1]);}
  MAs T query(int l, int r, As... ps){
```

```
    return S::op(v[r+1].query(ps...),S::inv(v[l].query(ps...)))
        ;
  } // eac6a8
}; // 4f6c04
template<class S> struct Psum<0, S>{
  using T = typename S::T;
  T val=S::id;
  void set(T x){val=x;}
  void push(Psum& a){val=S::op(a.val,val);}
  void init(){}
  T query(){return val;}
}; // 713694
```

Dist.h

Description: Disjoint Sparse Table Requires Monoid (op, id)
Memory: $\mathcal{O}(N \log N)$
Time: $\mathcal{O}(\log N)$

2f4715, 21L, 4m30s

```
#define repinv(i, a, b) for(int i = (a); i >= (b); i--)
template<class S> struct DiST{
  using T = typename S::T;
  int n, h; vector<vector<T>> t;
  int lg(signed x){return __builtin_clz(1)-__builtin_clz(x);}
  DiST(vector<T> v): n(sz(v)), h(lg(n)){
    if (n != (1<<h))n = 1<<(++h);
    t.assign(h, vector<T>(n)); v.resize(n, S::id);
    for(int d = 0, s = 1; d < h; d++, s *= 2)
    for(int m = s; m < n; m += 2*s){
      t[d][m] = v[m]; t[d][m-1] = v[m-1];
      rep(i, m+1, m+s)t[d][i] = S::op(t[d][i-1], v[i]);
      repinv(i, m-2, m-s)t[d][i] = S::op(v[i], t[d][i+1]);
    } // 3b44fe
  } // 1c2aa0
  T query(int l, int r){
    if (l==r)return t[0][l];
    int k = lg(l^r);
    return S::op(t[k][l], t[k][r]);
  } // 07c10a
}; // 612c6a
```

SparseTable.h

Description: Sparse Table Requires Idempotent Monoid S (op, inv, id)
Memory: $\mathcal{O}(n \log n)$
Time: $\mathcal{O}(1)$ query, $\mathcal{O}(n \log n)$ build

276609, 14L, 2m50s

```
template<class S> struct SpTable{
  using T = typename S::T;
  int n; vector<vector<T>> tab;
  int lg(signed x){return __builtin_clz(1)-__builtin_clz(x);}
  SpTable(vector<T> v):n(sz(v)),tab(1+lg(n),vector<T>(n,S::id))
      {
    rep(i,0,n)tab[0][i] = v[i];
    rep(i,0,lg(n))rep(j,0,n-(1<<i))
      tab[i+1][j] = S::op(tab[i][j], tab[i][j+(1<<i)]);
  } // c105d7
  T query(int l, int r){
    int k = lg(++r-l);
    return S::op(tab[k][l], tab[k][r-(1<<k)]);
  } // e06689
}; // 276609
```

SqrtDecomp.h

Description: Sqrt Decompostion
Memory: $\mathcal{O}(n)$
Time: $\mathcal{O}(n)$ build, $\mathcal{O}(sqrt(n))$ queries

f45235, 41L, 7m40s

```
struct SqrtDecomp {
  using K = ll; // single element information
  using T = ll; // block information
  int n, bsz, n_block;
  vector<T> v;
  vector<int> id;
  vector<K> block;
  SqrtDecomp(const vector<T> & x): n(sz(x)), v(x), id(n) {
    bsz = sqrt(n) + 1;
    n_block = (n + bsz - 1) / bsz; // ceil(n, bsz)
    rep(i, 0, n) id[i] = i / bsz;
    // Add information to block
    block = vector<K>(n_block, oo);
    rep(i, 0, n) block[id[i]] = min(block[id[i]], v[i]);
  } // 3bc167
  void update(int idx, ll x) { // Update set idx to x
    int bid = id[idx];
    block[bid] = oo;
    v[idx] = x;
    rep(i, bid * bsz, min((bid+1)*bsz, n)) block[bid] = min(
        block[bid], v[i]);
  } // 7aff89
  ll query(int l, int r) { // Query of min in interval [l, r]
    assert(l <= r); // Or return id;
    ll ans = oo;
    auto sblk = [&](int bid, int flag) { // flag [left, right,
        both]
      rep(i, max(l, bid*bsz), min((bid+1)*bsz, r+1)) ans = min(
          ans, v[i]);
    }; // f49504
    auto allblk = [&](int bid) { // Solve entire block
      ans = min(ans, block[bid]);
    }; // 3566fc
    if (id[l] == id[r]) {
      sblk(id[l], 2);
    } // 340382
    else {
      sblk(id[l], 0);
      rep(i, id[l]+1, id[r]) allblk(i);
      sblk(id[r], 1);
    } // e1769a
    return ans;
  } // 7a0d23
}; // f45235
```

6.2.1 Segment Tree

SegmentTree.h

Description: Half-Open Iterative SegTree
Time: $\mathcal{O}(\log N)$

0ea399, 17L, 2m50s

```
template<class S> struct SegTree{
  using T = typename S::T;
  int n; vector<T> seg;
  SegTree(int s):n(s),seg(2*n,S::id){}
  void update(T x, int i){
    for(seg[i+=n]=x;i/=2;)
      seg[i] = S::op(seg[2*i], seg[2*i+1]);
  } // 781838
  T query(int l, int r){ // [l, r)
    T vl=S::id, vr=S::id;
    for(l+=n,r+=n;l<r;l/=2,r/=2){
      if (l&1)vl = S::op(vl, seg[l++]);
      if (r&1)vr = S::op(seg[--r], vr);
    } // eabcd5
    return S::op(vl, vr);
  } // 25936a
}; // 0ea399
```

LazySegmentTree.h

Description: Lazy Seg (half-open), N must be a power of 2. Can be transformed into Seg Beats by uncommenting conditions.
Time: $\mathcal{O}(\log N * (ch + cmp))$.

fa8696, 30L, 8m

```
template<class S> struct SegBeats{ // n MUST be a power of 2
  using T = typename S::T; using L = typename S::L;
  int n; vector<T> seg; vector<L> lz; vector<bool> ig;
  SegBeats(int s):n(s),seg(2*n,S::id),lz(2*n),ig(2*n,1){}
  void apply(int p, L v, int l, int r){
    seg[p] = S::ch(seg[p],v,l,r);
    if (r-l>1)lz[p] = ig[p] ? v : S::cmp(lz[p], v), ig[p] = 0;
  } // f453ff
  void prop(int p, int l, int r){
    if (ig[p])return;
    int m = (l+r)/2; ig[p] = 1;
    apply(2*p, lz[p], l, m);
    apply(2*p+1, lz[p], m, r);
  } // f09d8e
  void update(L v, int l, int r){return update(v,l,r,1,0,n);}
  void update(L v, int lq, int rq, int no, int lx, int rx){
    if (rq <= lx or rx <= lq /*or S::brk(v,seg[no])*/)return;
    if (lq <= lx and rx <= rq /*and S::tag(v,seg[no])*/)return
        apply(no, v, lx, rx);
    int mid = (lx+rx)/2; prop(no,lx,rx);
    update(v,lq,rq,2*no,lx,mid); update(v,lq,rq,2*no+1,mid,rx);
    seg[no] = S::op(seg[2*no],seg[2*no+1]);
  } // 04eed2
  T query(int l, int r){return query(l,r,1,0,n);}
  T query(int lq, int rq, int no, int lx, int rx){
    if (rq <= lx or rx <= lq)return S::id;
    if (lq <= lx and rx <= rq)return seg[no];
    int mid = (lx+rx)/2; prop(no,lx,rx);
    return S::op(query(lq,rq,2*no,lx,mid),query(lq,rq,2*no+1,
        mid,rx));
  } // fb0086
}; // fa8696
```

MSegTree.h

Description: Multidimensional SegTree Requires Monoid (op, id)
Memory: $\mathcal{O}\left(N^D\right)$
Time: $\mathcal{O}\left((\log N)^D\right)$

e335e5, 26L, 5m40s

```
#define MAs template<class... As> //multiple arguments
template<int D, class S> struct MSegTree{
  using T = typename S::T;
  int n; vector<MSegTree<D-1, S>> seg;
  MAs MSegTree(int s, As... ds):n(s),seg(2*n, MSegTree<D-1, S>(
      ds...)){}
  MAs T get(int p, As... ps){return seg[p+n].get(ps...);}
  MAs void update(T x, int p, As... ps){
    seg[p+=n].update(x, ps...);
    while(p/=2)seg[p].update(S::op(seg[2*p].get(ps...),seg[2*p
        +1].get(ps...)), ps...);
  } // 2c8b52
  MAs T query(int l, int r, As... ps){
    T lv=S::id,rv=S::id;
    for(l+=n,r+=n+1;l<r;l/=2,r/=2){
      if (l&1)lv = S::op(lv,seg[l++].query(ps...));
      if (r&1)rv = S::op(seg[--r].query(ps...),rv);
    } // 1569b6
    return S::op(lv,rv);
  } // bc7474
}; // da06ff
template<class S> struct MSegTree<0, S>{
  using T = typename S::T;
  T val=S::id;
  T get(){return val;}
```

```
  void update(T x){val=x;} //set update!
  T query(){return val;}
}; // 50402e
```

LazySparseSeg.h

Description: Lazy Sparse Seg (half-open). Lazy can be removed (prop, lz, ig, ch/cmp)
Time: $\mathcal{O}(\log N * (ch + cmp))$.

fd0af0, 37L, 9m40s

```
template<class I, class S> struct LazySparseSeg{ //I is index
    type
  using T = typename S::T; //value type
  using L = typename S::L; //update type
  struct Node{int lc, rc; T val; L lz; bool ig;};
  I n; vector<Node> v;
  int new_node(){return v.eb(0,0,S::id,L(),1), sz(v)-1;}
  LazySparseSeg(I s): n(s){
    //v.reserve(MXN); //faster node creation
    new_node(); new_node(); //blank and root node
  } // c4ba1b
  void apply(int i, L x, I lx, I rx){
    v[i].val = S::ch(v[i].val, x, lx, rx);
    if (rx-lx>1)v[i].lz = v[i].ig ? x : S::cmp(v[i].lz, x), v[i
        ].ig = 0;
  } // 9ccb90
  void prop(int i, I lx, I rx){
    if (!v[i].lc)v[i].lc = new_node(), v[i].rc = new_node();
    if (v[i].ig)return;
    I mx = (lx+(rx-lx)/2); v[i].ig = 1;
    apply(v[i].lc, v[i].lz, lx, mx); apply(v[i].rc, v[i].lz, mx
        , rx);
  } // a00350
  void update(L x, I l, I r){return update(x, l, r, 1, 0, n);}
  void update(L x, I l, I r, int i, I lx, I rx){
    if (r <= lx or rx <= l)return;
    if (l <= lx and rx <= r)return apply(i, x, lx, rx);
    I mx = (lx+(rx-lx)/2); prop(i, lx, rx);
    int lc = v[i].lc, rc = v[i].rc;
    update(x, l, r, lc, lx, mx); update(x, l, r, rc, mx, rx);
    v[i].val = S::op(v[lc].val, v[rc].val);
  } // f721de
  T query(I l, I r){return query(l, r, 1, 0, n);}
  T query(I l, I r, int i, I lx, I rx){
    if (r <= lx or rx <= l)return S::id;
    if (l <= lx and rx <= r)return v[i].val;
    I mx = (lx+(rx-lx)/2); prop(i, lx, rx);
    return S::op(query(l, r, v[i].lc, lx, mx), query(l, r, v[i
        ].rc, mx, rx));
  } // 52a49e
}; // fd0af0
```

LazyPersistentSeg.h

Description: Persistent Lazy Sparse Segment Tree Can be changed by modifying Spec
Time: $\mathcal{O}(\log N * (ch + cmp))$

b43ed0, 39L, 10m40s

```
template<class I, class S> struct LazyPersistentSeg{ //I is
    index type
  using T = typename S::T; //value type
  using L = typename S::L; //lazy type
  struct Node{int lc, rc; T val; L lz; bool ig;};
  I n; vector<Node> v;
  int new_node(int l=0, int r=0){return v.eb(l,r,S::op(v[l].val
      ,v[r].val),L(),1), sz(v)-1;}
  LazyPersistentSeg(){ //only creates object, should be "init"
      ed to get root
    //v.reserve(MXN); //faster node creation
    v.eb(0,0,S::id,L(),1); //blank node
  } // 00247f
  int init(I s){return n = s, new_node();}
  int lazy_clone(int i, L lz, I lx, I rx){
    int ni = new_node(v[i].lc, v[i].rc);
    v[ni].lz = v[i].ig ? lz : S::cmp(v[i].lz, lz);
    v[ni].ig = 0; v[ni].val = S::ch(v[i].val, lz, lx, rx);
    return ni;
  } // 4c0efb
  void prop(int i, I lx, I rx){
    if (v[i].ig)return;
    int mx = lx + (rx - lx) / 2; v[i].ig = 1;
    if (lx < rx)
      v[i].lc = lazy_clone(v[i].lc, v[i].lz, lx, mx),
      v[i].rc = lazy_clone(v[i].rc, v[i].lz, mx, rx);
  } // 551bde
  int update(L lz, I l, I r, int root){return update(lz, l, r,
      root, 0, n);}
  int update(L lz, I l, I r, int i, I lx, I rx){
    if (r <= lx or rx <= l)return i;
    if (l <= lx and rx <= r)return lazy_clone(i, lz, lx, rx);
    I mx = lx + (rx - lx) / 2; prop(i, lx, rx);
    return new_node(update(lz, l, r, v[i].lc, lx, mx), update(
        lz, l, r, v[i].rc, mx, rx));
  } // ca252e
  T query(I l, I r, int root){return query(l, r, root, 0, n);}
  T query(I l, I r, int i, I lx, I rx){
    if (r <= lx or rx <= l)return S::id;
    if (l <= lx and rx <= r)return v[i].val;
    I mx = lx + (rx - lx) / 2; prop(i, lx, rx);
    return S::op(query(l, r, v[i].lc, lx, mx), query(l, r, v[i
        ].rc, mx, rx));
  } // 893689
}; // b43ed0
```

Strings (7)

Hashing.h

Description: String hashing (multiple mods and 2^{32})
Memory: $\mathcal{O}(n)$
Time: $\mathcal{O}(1)$ query, $\mathcal{O}(n)$ build

c40ee3, 23L, 5m40s

```
typedef uint64_t ull;
template<int M, class B> struct A {
  int x; B b; A(int x=0) : x(x), b(x) {}
  A(int x, B b) : x(x), b(b) {}
  A operator+(A o) const {int y = x+o.x; return{y - (y>=M)*M, b
      +o.b};}
  A operator-(A o) const {int y = x-o.x; return{y + (y< 0)*M, b
      -o.b};}
  A operator*(A o) const { return {(int)((ll)x*o.x % M), b*o.b}
      ; }
  explicit operator ull() const { return x ^ (ull) b << 21; }
}; // e03276
typedef A<1000000007, A<1000000009, unsigned>> H;
static int C; // initialize to a number less than MOD or random
struct HashInterval {
  int n; vector<H> ha, pw;
  template<typename S>
  HashInterval(const S & str) : n(sz(str)), ha(n+1), pw(n+1) {
    pw[0] = 1;
    rep(i,0,n)
      ha[i+1] = ha[i] * C + str[i],
      pw[i+1] = pw[i] * C;
  } // 185a86
  H query(int a, int b) { return ha[b] - ha[a] * pw[b - a]; }
  H queryI(int a, int b) { return query(n - b, n - a); }
}; // 434a8c
```

KMP.h

Description: KMP automaton
Memory: $\mathcal{O}(N)$
Time: $\mathcal{O}(N)$ build, $\mathcal{O}(1)$ query (amortized)

8f8450, 19L, 4m

```
struct KMP {
  string P; int n; vector<int> nb;
  KMP(string& p) : P(p), n((int)P.size()), nb(n+1) {
    for(int k = 1; k < n; k++) nb[k+1] = nxt(nb[k], P[k]);
  } // ca6dc8
  int nxt(int i, char c){
    for(; i; i = nb[i])if (i < n and P[i]==c)return i+1;
    return P[0]==c;
  } // 2b99e2
  vector<vector<int>> dfa;
    void build_dfa(){
    dfa.assign(n+1, vector<int>(26));
        dfa[0][P[0]-'a'] = 1; //only way to advance at 0
        for(int k = 1; k <= n; k++)
            for(int c = 0; c < 26; c++)
                if (k < n and P[k] == 'a'+c) dfa[k][c] = k+1;
                else dfa[k][c] = dfa[nb[k]][c];
    } // f47d83
}; // 8f8450
```

Zfunc.h

Description: z[i] computes the length of the longest common prefix of s[i:] and s, except z[0] = 0. (abacaba -> 0010301)
Time: $\mathcal{O}(n)$

ee09e2, 9L, 1m30s

```
vi Z(const string& S) {
  vi z(sz(S)); int l = -1, r = -1;
  rep(i,1,sz(S)) {
    z[i] = i >= r ? 0 : min(r - i, z[i - l]);
    while (i + z[i] < sz(S) && S[i + z[i]] == S[z[i]]) z[i]++;
    if (i + z[i] > r) l = i, r = i + z[i];
  } // 44be47
  return z;
} // ee09e2
```

Manacher.h

Description: For each position in a string, computes p[0][i] = half length of longest even palindrome around pos i, p[1][i] = longest odd (half rounded down).
Time: $\mathcal{O}(N)$

e7ad79, 11L, 2m10s

```
array<vi, 2> manacher(const string& s) {
  int n = sz(s); array<vi,2> p = {vi(n+1), vi(n)};
  rep(z,0,2) for (int i=0,l=0,r=0; i < n; i++) {
    int t = r-i+!z;
    if (i<r) p[z][i] = min(t, p[z][l+t]);
    int L = i-p[z][i], R = i+p[z][i]-!z;
    while (L>=1 && R+1<n && s[L-1] == s[R+1]) p[z][i]++, L--, R
        ++;
    if (R>r) l=L, r=R;
  } // a843d3
  return p;
} // e7ad79
```

MinRotation.h

Description: Finds the lexicographically smallest rotation of a string.
Usage: `rotate(v.begin(), v.begin()+minRotation(v), v.end());`
Time: $\mathcal{O}(N)$

d07a42, 8L, 1m20s

```
int minRotation(string s) {
  int a=0, N=sz(s); s += s;
  rep(b,0,N) rep(k,0,N) {
    if (a+k == b || s[a+k] < s[b+k]) {b += max(0, k-1); break;}
    if (s[a+k] > s[b+k]) { a = b; break; }
```

```
  } // 9374b1
  return a;
} // d07a42
```

Automaton.h

Description: Suffix automaton with parameterized transition table (can be array or map)
Memory: $\mathcal{O}(n * |G|)$
Time: $\mathcal{O}(n)$ build

70eb49, 30L, 5m10s

```
template<class C, class G> struct Automaton{
  vector<G> dag;
  vector<int> lnk, len, cnt;
  int last;
  template<class S> Automaton(S& str):
    dag(2),lnk(2,0),len(2,0),cnt(2,0),last(1){
    for(C& c : str)add(c); //can do .reserve(2*n+2)
  } // cd5999
  int new_node(int l, int is_pref){
    dag.pb(G()); lnk.pb(0); len.pb(l); cnt.pb(is_pref);
    return sz(dag)-1;
  } // 0814f6
  void add(C& c){
    int cur = new_node(len[last]+1, 1), p = last;
    while(p and not dag[p][c])dag[p][c] = cur, p = lnk[p];
    if (not p)lnk[cur] = 1;
    else {
      int q = dag[p][c];
      if (len[p]+1 == len[q])lnk[cur] = q;
      else{
        int clone = new_node(len[p]+1, 0);
        dag[clone] = dag[q];
        lnk[clone] = lnk[q];
        lnk[q] = lnk[cur] = clone;
        while(p and dag[p][c] == q)dag[p][c]=clone, p=lnk[p];
      } // 270a8a
    } // 9e6a19
    last = cur;
  } // b1dc88
}; // 70eb49
```

Aho.h

Description: Aho-Corasick
Memory: $\mathcal{O}(n * 26)$
Time: $\mathcal{O}(n)$ build

ccf3a0, 47L, 6m40s

```
const int K = 26;
struct Vertex {
    int next[K];
    bool output = false;
    int p = -1;
    char pch;
    int link = -1;
    int go[K];

    Vertex(int _p=-1, char ch='$') : p(_p), pch(ch) {
        fill(begin(next), end(next), -1);
        fill(begin(go), end(go), -1);
    } // 2636d9
}; // 0c589a

vector<Vertex> t(1);

void add_string(string const& s) {
    int v = 0;
    for (char ch : s) {
        int c = ch - 'a';
        if (t[v].next[c] == -1) {
            t[v].next[c] = sz(t);
            t.eb(v, ch);
        } // 2abd5c
        v = t[v].next[c];
    } // fc1e0d
    t[v].output = true;
} // 1bb4a6

int go(int v, char ch);
int get_link(int v) {
    if (t[v].link == -1) {
        if (v == 0 || t[v].p == 0) t[v].link = 0;
        else t[v].link = go(get_link(t[v].p), t[v].pch);
    } // e9856f
    return t[v].link;
} // 1e38ab

int go(int v, char ch) {
    int c = ch - 'a';
    if (t[v].go[c] == -1) {
        if (t[v].next[c] != -1) t[v].go[c] = t[v].next[c];
        else t[v].go[c] = v == 0 ? 0 : go(get_link(v), ch);
    } // 202af4
    return t[v].go[c];
} // e442d8
```

DP (8)

We don't know DP optimizations yet :/

Various (9)

9.1 Random

RNG.h

Description: RNGs
Time: $\mathcal{O}(1)$

6823af, 18L, 3m50s

```
mt19937 rng(chrono::steady_clock::now().time_since_epoch().
    count()); // mt19937_64
void example(){
    int n; cin >> n;
    uniform_int_distribution<int> distribution(1,n);
    int num = distribution(rng); // num no range [1, n]
    vector<int> vec(n);
    iota(vec.begin(), vec.end(), 1);
    shuffle(vec.begin(), vec.end(), rng); // shuffle
} // a3f379

using ull = unsigned long long;
ull mix(ull o){
    o+=0x9e3779b97f4a7c15;
    o=(o^(o>>30))*0xbf58476d1ce4e5b9;
    o=(o^(o>>27))*0x94d049bb133111eb;
    return o^(o>>31);
} // bc6211
ull hash(pii a) {return mix(a.first ^ mix(a.second));}
```

9.2 Ternary Search

GoldenSectionSearch.h

Description: Finds the argument minimizing the function f in the interval $[a, b]$ assuming f is unimodal on the interval, i.e. has only one local minimum and no local maximum. The maximum error in the result is *eps*. Works equally well for maximization with a small change in the code. See TernarySearch.h in for a discrete version.
Usage: `double func(double x) { return 4+x+.3*x*x; }`
`double xmin = gss(-1000,1000,func);`
Time: $\mathcal{O}(\log((b-a)/\epsilon))$

31d45b, 16L, 3m20s

```
// It is important for r to be precise,
// otherwise we don't necessarily maintain the inequality a <
    x1 < x2 < b.
double gss(double a, double b, double (*f)(double)) {
  double r = (sqrt(5)-1)/2, eps = 1e-7;
  double x1 = b - r*(b-a), x2 = a + r*(b-a);
  double f1 = f(x1), f2 = f(x2);
  while (b-a > eps)
    if (f1 < f2) { //change to > to find maximum
      b = x2; x2 = x1; f2 = f1;
      x1 = b - r*(b-a); f1 = f(x1);
    } else { // 4513d0
      a = x1; x1 = x2; f1 = f2;
      x2 = a + r*(b-a); f2 = f(x2);
    } // 2fe74a
  return a;
} // 31d45b
```

TernarySearch.h

Description: Find the smallest i in $[a, b]$ that maximizes $f(i)$, assuming that $f(a) < \ldots < f(i) \geq \ldots \geq f(b)$. To reverse which of the sides allows non-strict inequalities, change the $<$ marked with (A) to $<=$, and reverse the loop at (B). To minimize f, change it to $>$, also at (B).
Usage: `int ind = ternSearch(0,n-1,[&](int i){return a[i];});`
Time: $\mathcal{O}(\log(b-a))$

afbac9, 9L, 1m30s

```
template<class F> int ternSearch(int a, int b, F f) {
  while (b - a >= 5) {
    int mid = (a + b) / 2;
    if (f(mid) < f(mid+1)) a = mid; // (A)
    else b = mid+1;
  } // ce7859
  rep(i,a+1,b+1) if (f(a) < f(i)) a = i; // (B)
  return a;
} // afbac9
```

9.3 Optimization tricks

9.3.1 Bit hacks

- `x & -x` is the least bit in `x`.
- `for (int x = m; x; ) { --x &= m; ... }` loops over all subset masks of `m` (except `m` itself).
- `c = x&-x, r = x+c; (((r^x) >> 2)/c) | r` is the next number after `x` with the same number of bits set.
- `rep(b,0,K) rep(i,0,(1 << K))`
  `if (i & 1 << b) D[i] += D[i^(1 << b)];`
  computes all sums of subsets.

9.3.2 Pragmas

- `#pragma GCC optimize ("Ofast")` will make GCC auto-vectorize loops and optimizes floating points better.
- `#pragma GCC target ("avx2")` can double performance of vectorized code, but causes crashes on old machines.
- `#pragma GCC optimize ("bmi,bmi2,lzcnt,popcnt")` are good for bitsets.